

Inno Agent 使用说明书

版本 v0.2.3 · 发布日期 2026-06-05 · 华东师范大学上海智能教育研究院

前言

本文档是 Inno Agent 的完整使用教程，适合希望将 Inno Agent 部署并深度使用的个人学习者或研究者。

文档导读

章节	适合场景
第 1 章	想了解设计理念和架构，再决定要不要用
第 2—3 章	第一次安装，或者安装遇到问题
第 4—5 章	想让 Agent 记住自己、管理知识库
第 6—9 章	想解锁定时任务、飞书、终端、Skills 扩展
第 10 章	需要部署到服务器或生产环境
附录	API、配置字段快速查找

第 1 章 认识 Inno Agent

学完本章，你将理解 Inno Agent 的设计定位和核心架构，能够说清楚它与通用 LLM Agent 的本质区别，以及三层记忆各自负责什么。

学习目标

- 理解个人学习 Agent 的独特优化目标，以及为什么通用编程 Agent 不适合直接用于学习
- 掌握三层记忆架构（L1/L2/L3）的职责划分、边界规则和更新机制
- 通过一次完整的对话流程，理解各组件如何协作
- 了解 Inno Agent 的四个交互入口及其适用场景

1.1 什么是 Inno Agent

Inno Agent 是一个面向个人学习者的开源 AI 辅导系统，基于 Pi 编程 Agent SDK 构建。它通过 Pi 的扩展机制（Extension）接入学习专属工具，而不修改 SDK 内核。

它的核心主张是：个性化学习所需的记忆、调度与反馈机制，不应藏在对话历史里，而应作为系统的一等公民明确管理。

举一个具体的对比：

- 如果你用通用 AI 助手学习「Transformer 注意力机制」，第二天再打开新对话，它完全不记得你昨天学到哪、哪个地方还没弄清楚、你偏好用类比来理解数学公式——你得重新解释一遍背景才能继续。
- 用 Inno Agent，每次对话开始前，系统自动读取你的学习档案，知道你目前掌握了 self-attention 但还没理解 multi-head，偏好先看代码再看公式，上次误解了「Q/K/V 必须来自不同输入」——它直接从这里继续，无需你重新铺垫。

这个持续跟踪通过三层独立记忆系统实现，而不是依靠越来越长的对话历史。

Inno Agent 提供两种运行模式：

- **终端 CLI**：纯命令行 TUI 交互，轻量、无 HTTP 服务，适合本地单机或服务器使用
- **Web UI**：基于 React 19 + Tailwind 4 的三栏学习工作台，有完整图形界面、内嵌终端、知识图谱和学习档案，由 Node.js HTTP 服务器后端支撑

两种模式共享同一套运行时目录（`runtime/` 与 `workspace/`），配置、会话、记忆、Skills 完全对齐——CLI 里积累的学习记录，在 Web UI 里同样可见。

1.2 为什么不直接用通用 Agent

通用编程 Agent 的核心能力是处理大规模代码库、执行长流程任务。这使得它们的设计取向是：最大的模型 + 最长的上下文窗口 + 完整的工具链。这套取向在教育场景里会产生几个具体问题：

问题一：成本与延迟不适合高频辅导。 一次学习可能涉及 20—50 轮问答。200k token 上下文窗口的云端大模型在这种高频轮次下成本高、响应延迟大。辅导本质上是「轮次紧凑的即时反馈」，而不是「生成大段代码块」。

问题二：记忆模式根本不匹配学习需求。 通用 Agent 的「记忆」是上下文窗口里的对话历史。这对学习有根本性缺陷：学习状态（你掌握了什么、误解了什么）需要长期积累；但对话历史在压缩时会丢失重要信号；积累几十次对话后，Agent 无法区分哪些是稳定的学习事实、哪些是一次性讨论。

问题三：无法主动规划和推送。 通用 Agent 响应用户输入，但不会在你没打开对话时主动发「今天该复习 X 了」。学习的很大一部分价值来自主动的间隔重复和复盘，这需要后台调度机制。

维度	通用编程 Agent	Inno Agent
主要任务	开放式软件工程	概念讲解、错误诊断、习题生成
交互节奏	长流程、批量执行	轮次紧凑、即时反馈
记忆机制	上下文窗口对话历史	三层独立记忆（稳定画像 + 知识库 + 近期对话）
隐私敏感	代码仓库内容	学习者个人状态、误解记录
运行形态	云端交互为主	支持本地模型，所有数据本地存储
主动性	被动响应	定时任务主动推送复习提醒

Inno Agent 的目标是：用一个较小的本地化模型，通过外置的记忆和工具脚手架，提供可实用的个性化辅导——而不是依赖最强的云端大模型。

1.3 三层记忆架构

Inno Agent 将学习状态分为三个独立的记忆层，每层有不同的生命周期、写入规则和读取时机：

L1 学习者画像 (Learner Profile)

— 生命周期：长期，跨会话持久存在

— 写入时机：Agent 记录学习事件 → 确定性规则自动更新画像

— 读取时机：每轮对话开始前自动读取，构建上下文包注入

— 内容：学习目标 / 概念掌握度 (0-1) / 已识别误解 / 教学偏好

— 文件：data/learner/profile.json + events.jsonl

L2 知识 Wiki (Native Wiki Memory)

— 生命周期：长期，随归档操作增长，人类可直接阅读和编辑

— 写入时机：用户上传文档或 Agent 调用 l2_archive 触发摄入

— 读取时机：Agent 按需调用 l2_query 检索，不自动全量注入

— 内容：来源摘要页 / 实体页 / 概念页 / 分析页 (Markdown + YAML frontmatter)

— 文件：data/l2/wiki/

L3 会话历史 (Session Records)

— 生命周期：中短期，高频变化，Pi SDK 自动管理压缩

— 写入时机：每轮对话自动追加

— 读取时机：Pi SDK 自动拼接到上下文窗口

— 内容：近期对话轮次 / 工具调用记录 / 压缩快照

— 文件：data/sessions/ (Pi SDK 负责管理)

为什么要三层分离？

三层分离不是为了架构上的「优雅」，而是解决一个实际问题：不同类型的学习信息有完全不同的访问模式和生命周期。

- **学习者的目标和掌握状态**（L1）变化很慢，但必须在每次对话中都用上，不能每次都从历史里重新推断。
- **学习材料的内容**（L2）量大，不能全量塞进上下文，但需要时必须能精确找到。
- **最近的对话**（L3）对当前这轮推理有价值，但一周后的价值接近零，不应污染稳定的学习档案。

把三类信息混在一个上下文窗口里，要么撑爆 token 预算，要么在压缩时把重要的学习状态丢掉。

三层边界规则（写入系统提示词，Agent 必须遵守）：

信息类型	写到哪层	触发工具
学习者声明了新目标	L1	<code>record_learning_event</code> (<code>goal_declared</code> 事件)
完成了一次概念讲解	L1	<code>record_learning_event</code> (<code>concept_explained</code> 事件，掌握度 +0.02)
做了练习，答对/答错	L1	<code>record_learning_event</code> (<code>exercise_attempt</code> 事件，掌握度 +0.03)
识别到一个误解	L1	<code>record_learning_event</code> (<code>misconceptions</code> 字段更新)
用户上传了一篇论文/笔记	L2	<code>l2_archive</code>
讨论了某个话题的细节	L3	自动追加，无需工具调用
一次性闲聊、情绪表达	不写入任何持久层	—

1.4 上下文包（Context Pack）：记忆如何进入每轮对话

L1 记忆不是把整个 `profile.json` 直接塞进上下文，而是每轮对话开始前通过 `before_agent_start` 钩子动态构建一个精简上下文包，只取当前最相关的部分：

```
interface LearnerContextPack {
  active_goal?: string;           // 当前最高优先级目标
  relevant_concepts: {           // 掌握度最低的 5 个相关概念（升序）
    // ...
  }
```

```

    concept_id: string;
    mastery: number;                // 0-1, 越低越弱
    diagnosis: string;
  }[];
  active_misconceptions: string[]; // 当前活跃的误解描述列表
  teaching_hints: string[];       // 从偏好设置转化的教学提示
  recent_events?: {               // 最近 8 条学习事件摘要
    event_id: string;
    event_type: string;
    timestamp: string;
    summary: string;
  }[];
  review_due_concepts?: {         // 今天到期需要复习的概念
    concept_id: string;
    review_due_at: string;
    mastery: number;
  }[];
}

```

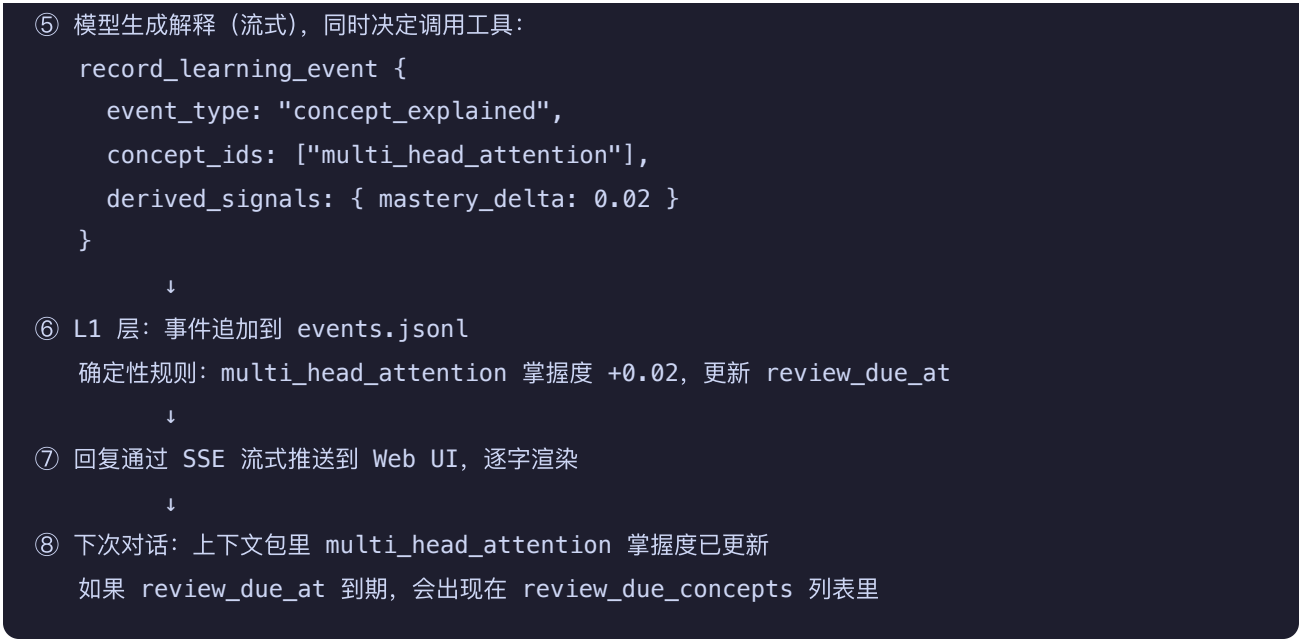
这个包被拼在系统提示词开头，格式是一段「学习背景」文本，告诉 Agent：你现在在教的这个学习者，目标是 X，目前最弱的是 Y 和 Z，有一个误解需要注意，偏好用例子先行……

设计效果：一个 35B 的本地模型，拿到这个精简的上下文包后，可以在不了解学习者完整历史的情况下，做出比较准确的个性化教学决策——因为决策所需的关键信息已经被系统预先整理好了。

1.5 一次完整对话的内部流程

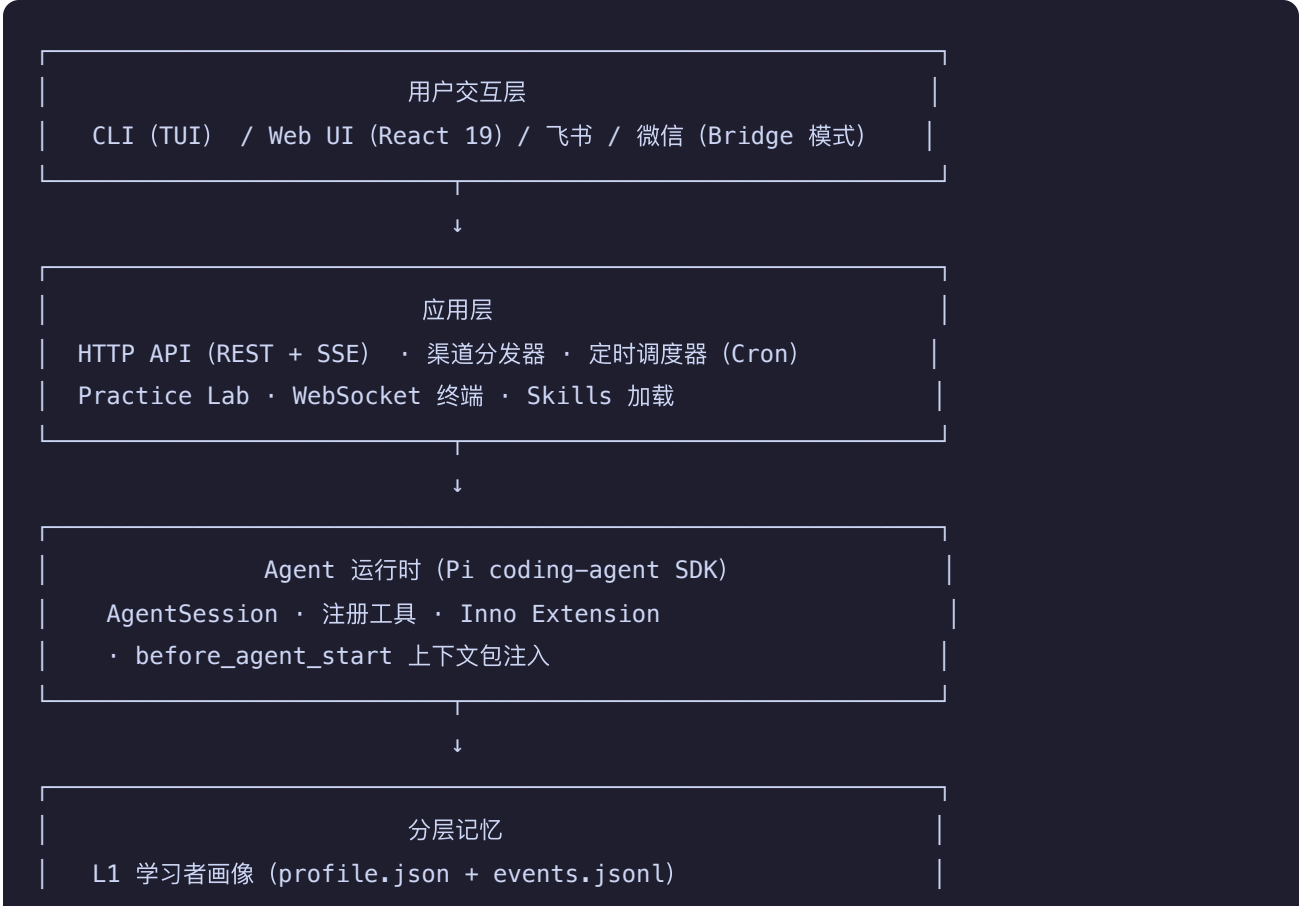
以「帮我解释 Transformer 的 multi-head attention」为例，说明各组件如何协作：

- ① 用户在 Web UI 输入消息并发送
 - ↓
- ② POST /api/chat/stream 进入 server.ts，找到当前会话的 Pi AgentSession
 - ↓
- ③ before_agent_start 钩子触发：
 - 读取 profile.json + 最近 8 条 events.jsonl
 - buildContextPack() 生成精简上下文包
 - 上下文包拼入系统提示词开头
 - ↓
- ④ Pi AgentSession 开始推理
 - 模型收到：[系统提示 + 上下文包] + [L3 历史对话] + [当前用户消息]
 - ↓



整个过程中，L2（知识 Wiki）仅在模型主动调用 `l2_query` 时介入（即用户问题需要检索已归档资料时）；L3 在每轮结束后由 Pi SDK 自动追加，无需 Inno 层干预。

1.6 系统架构总览



```
| L2 原生 Wiki (data/l2/wiki/ Markdown 文件) |
| L3 Pi SessionManager (sessions/ 会话历史) |
|_____|
```

1.7 注册工具列表

Inno Extension 在每次会话启动时向 Pi 运行时注册以下工具。Agent 只能通过这些工具改变系统状态：

工具组	工具名称	作用
L1 学习者	get_learner_context	读取完整学习者画像
	record_learning_event	记录学习事件，触发掌握度/误解自动更新
	patch_learner_profile	直接修改画像中的单个字段
	update_learner_profile	批量更新画像
	review_learner_profile	生成当前画像摘要（供 Agent 核对）
L2 Wiki	l2_archive	归档文档到 L2 知识库，触发摘要和链接生成
	l2_query	按关键词检索 L2，返回相关页面内容
文档	parse_document	解析工作区内的文档文件为 Markdown
调度器	create_scheduled_job	创建定时任务（Cron 表达式 + 任务类型）
	list_scheduled_jobs	列出所有定时任务
	update_scheduled_job	修改定时任务配置
	delete_scheduled_job	删除定时任务
	run_scheduled_job	立即手动触发一次指定任务
Practice Lab	create_practice_lab	在工作区创建代码练习文件和 README
交互	ask_user_question	向用户提问（需要用户补充信息时使用）

小结

- Inno Agent 的核心优势是三层独立记忆和主动调度，而不是最大的模型或最长的上下文

- L1 每轮自动注入（精简上下文包），L2 按需检索（不自动全量注入），L3 由 SDK 托管——三层访问模式完全不同
- 上下文包机制让小模型也能做出有效的个性化教学决策
- 所有持久化均为本地文件（JSON / JSONL / Markdown），无外部数据库依赖，可完全离线运行
- 下一章我们实际安装并启动，第一次对话后可以在学习档案里看到 Agent 自动记录的第一条学习事件

第 2 章 安装与首次运行

学完本章，你将拥有一个可以正常对话的 Inno Agent 运行环境，并理解配置文件的完整结构。

学习目标

1. 安装符合要求的 Node.js 环境（涵盖 macOS 和 Windows 平台）
2. 完成项目依赖安装和构建，理解每步的预期输出
3. 配置 `config.json`，连接至少一个模型 API
4. 成功启动 Web 服务器，完成第一次对话，确认 L1 记忆记录了学习事件

2.1 环境要求

Inno Agent 后端是 Node.js 应用，前端构建也依赖 Node.js。

依赖	最低版本	验证命令
Node.js	20.6.0	<code>node --version</code>
npm	随 Node.js 附带，无需单独安装	<code>npm --version</code>
Git	推荐，用于后续更新代码	<code>git --version</code>

在终端运行以下命令验证：

```
node --version
# 应输出 v20.x.x 或更高，例如 v22.3.0

npm --version
# 应输出 10.x.x 或更高
```


【截图占位】 此处插入截图——终端分别运行 `node --version` 和 `npm --version`，显示版本号符合要求

如果 `node --version` 提示「命令未找到」或版本低于 20.6.0，参考以下方式安装：

macOS 推荐：使用 nvm (Node Version Manager)

nvm 允许在同一台机器上管理和切换不同版本的 Node.js，推荐开发者使用：

```
# 第 1 步：安装 nvm
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh | bash

# 第 2 步：重新加载 shell 配置（或重新打开终端）
source ~/.zshrc    # zsh 用户 (macOS 默认)
source ~/.bashrc   # bash 用户

# 第 3 步：安装 Node.js 22 (LTS, 满足 ≥20.6.0 要求)
nvm install 22
nvm use 22

# 第 4 步：验证
node --version     # 应显示 v22.x.x
```

macOS 备选：Homebrew

```
brew install node
node --version
```

Windows：官网下载安装包

1. 前往 <https://nodejs.org> 下载 LTS 版本（.msi 安装包）
2. 运行安装包，全程默认选项
3. 关键：安装过程中确认勾选「Add to PATH」，否则命令行无法识别 node 命令
4. 安装完成后重新打开 PowerShell 验证：

```
node --version
npm --version
```

提示 (Windows)：如果安装后 PowerShell 仍提示「无法识别的命令」，需要关闭并重新打开

PowerShell，让系统的 PATH 环境变量刷新生效。

【截图占位】 此处插入截图——macOS 终端（或 Windows PowerShell）中 `node --version` 和 `npm --version` 输出正常版本号

2.2 获取项目代码

方式 A: Git Clone（推荐）

Git Clone 方式便于后续用 `git pull` 跟进代码更新：

```
git clone <仓库地址> inno-agent
cd inno-agent
```

克隆完成后，终端会显示类似：

```
Cloning into 'inno-agent'...
remote: Enumerating objects: ...
Receiving objects: 100% (xxxx/xxxx) ...
Resolving deltas: 100% ...
```

方式 B: 下载 ZIP

1. 访问项目仓库页面
2. 点击绿色 **Code** 按钮 → **Download ZIP**
3. 解压到本地，进入解压后的目录：

```
cd ~/projects/inno-agent # macOS/Linux 示例路径
```

注意：ZIP 方式安装后，目录里没有 `.git` 文件夹，无法用 `git pull` 更新。建议长期使用的话选 Git Clone 方式。

2.3 安装依赖

进入项目根目录（即包含 `package.json` 的目录）后，执行：

```
npm install
```

这一步安装所有工作区的依赖（后端 + 前端），包括 Pi SDK、TypeScript、React、Vite 等，根据网络速度通常需要 1—3 分钟。

安装完成的终端末尾会显示：

```
added 1234 packages, and audited 1235 packages in 45s
```

`npm warn deprecated` 之类的黄色警告通常不影响使用，可以忽略。

如果看到红色 `npm error` 且安装中止，最常见原因是网络访问 npm 源失败（国内常见），尝试：

```
# 使用国内镜像源
npm install --registry https://registry.npmmirror.com

# 如果仍然失败，清除缓存后重试
npm cache clean --force
npm install --registry https://registry.npmmirror.com
```

【截图占位】 此处插入截图——`npm install` 成功完成，终端末尾显示 `added xxxx packages`

2.4 构建项目

依赖安装完成后，执行构建：

```
npm run build
```

这一步会：

1. 用 TypeScript 编译后端代码 → `apps/innno-agent/dist/`
2. 用 Vite 构建前端代码 → `apps/innno-agent/web/dist/`

构建过程约需 20—60 秒，成功完成后终端末尾会显示：

```
✓ built in x.xxs
```

验证构建成功：

```
ls apps/innno-agent/dist/      # 应能看到 cli.js、server.js 等文件
ls apps/innno-agent/web/dist/  # 应能看到 index.html、assets/ 目录
```

【截图占位】 此处插入截图—— `npm run build` 成功完成后的终端输出，显示 `TypeScript` 编译成功和 `Vite` `✓ built in x.xxs`

2.5 准备运行时目录

Inno Agent 把所有运行时数据（配置、会话、记忆、Skills）集中放在一个 `home` 目录，与代码仓库完全分开。这样代码更新时不会覆盖你的个人学习数据。

```
mkdir -p runtime/config runtime/data runtime/skills workspace
cp config.example.json runtime/config/config.json
```

创建完成后的目录结构：

```
runtime/
├── config/
│   └── config.json    ← 唯一需要你手动编辑的文件
├── data/              ← 首次运行后自动填充（学习记录、Wiki 等）
└── skills/            ← 安装 Skill 后会在这里出现内容

workspace/             ← Agent 的工作目录（你的学习项目文件放这里）
```

2.6 配置 config.json（关键步骤）

用任意文本编辑器打开 `runtime/config/config.json`（VS Code、记事本均可）。

你需要告诉 Inno Agent：用哪个 AI 服务、API Key 是什么、服务地址是什么。

配置最简流程（3 步）：

第 1 步：找到 `"apiKey"` 字段，替换为你的密钥

```
"apiKey": "替换为你的 API Key"
```

第 2 步：确认 `"baseUrl"` 和 `"api"` 字段与你的服务商一致

```
"baseUrl": "服务商 API 地址",
"api": "anthropic-messages"    // 或 "openai-completions"
```

第 3 步：保存文件（macOS `Cmd+S`，Windows `Ctrl+S`）

各主流服务商配置示例

直接复制对应部分，替换 `config.json` 中 `providers` 对象里的内容：

InnoSpark (Anthropic 兼容，推荐中国大陆用户)

```
"innospark": {
  "baseUrl": "https://api.innospark.cn",
  "api": "anthropic-messages",
  "apiKey": "sk-ant-你的密钥",
  "models": [
    {
      "id": "claude-sonnet-4-6",
      "name": "Claude Sonnet 4.6",
      "contextWindow": 200000,
      "maxTokens": 16384
    },
    {
      "id": "claude-opus-4-8",
      "name": "Claude Opus 4.8",
      "contextWindow": 200000,
      "maxTokens": 32768
    }
  ]
}
```

Anthropic 官方 API

```
"anthropic": {
  "baseUrl": "https://api.anthropic.com",
  "api": "anthropic-messages",
  "apiKey": "sk-ant-你的密钥",
  "models": [
    {
      "id": "claude-sonnet-4-6",
      "name": "Claude Sonnet 4.6",
      "contextWindow": 200000,
      "maxTokens": 16384
    }
  ]
}
```

```
]
}
```

DeepSeek (推荐走 Anthropic 兼容端点, 思考链支持更好)

```
"deepseek": {
  "baseUrl": "https://api.deepseek.com/anthropic",
  "api": "anthropic-messages",
  "apiKey": "sk-你的密钥",
  "models": [
    {
      "id": "deepseek-v4-pro",
      "name": "DeepSeek V4 Pro",
      "contextWindow": 1000000,
      "maxTokens": 32768
    }
  ]
}
```

OpenAI 官方 API

```
"openai": {
  "baseUrl": "https://api.openai.com/v1",
  "api": "openai-completions",
  "apiKey": "sk-proj-你的密钥",
  "models": [
    {
      "id": "gpt-4o",
      "name": "GPT-4o",
      "contextWindow": 128000,
      "maxTokens": 16384
    }
  ]
}
```

本地 Ollama (完全离线, 无需 API Key)

前置: 已安装 Ollama 并拉取了模型, 如 `ollama pull qwen2.5:14b`

```
"ollama": {
  "baseUrl": "http://localhost:11434/v1",
  "api": "openai-completions",
  "apiKey": "ollama",
  "models": [
    {
      "id": "qwen2.5:14b",
      "name": "Qwen2.5 14B (本地)",
      "contextWindow": 32768,
      "maxTokens": 8192
    }
  ]
}
```

【截图占位】 此处插入截图——VS Code 或文本编辑器打开 config.json，高亮 apiKey 字段所在位置

api 字段填写规则——填错这个是最常见的错误：

服务商 / 场景	正确的 api 值
Anthropic 官方 API	anthropic-messages
InnoSpark	anthropic-messages
DeepSeek (/anthropic 端点)	anthropic-messages
DeepSeek (默认端点, 不含 /anthropic)	openai-completions
OpenAI 官方 API	openai-completions
OpenRouter	openai-completions
本地 Ollama	openai-completions
本地 LM Studio	openai-completions
本地 vLLM	openai-completions

简单记法：服务商文档说「兼容 Anthropic Messages API」，或地址含 /anthropic 路径，填 anthropic-messages；其他情况填 openai-completions。

同时配置多个 Provider（可选）

如果你有多个 API Key，可以在 `providers` 里同时配置多个，Web UI 中随时切换：

```
{
  "defaultProvider": "innospark",
  "defaultModel": "claude-sonnet-4-6",
  "providers": {
    "innospark": { ... },
    "deepseek": { ... },
    "ollama": { ... }
  }
}
```

`defaultProvider` 和 `defaultModel` 决定启动时默认使用哪个，之后可以在 Web UI 设置面板切换，无需重启服务。

2.7 启动 Web 服务器

配置文件保存好后，运行：

```
npm run server -- --home ./runtime --workspace ./workspace --port 3000
```

正常启动的终端日志大致如下：

```
[inno] loading config from ./runtime/config/config.json
[inno] data dir: ./runtime/data
[inno] skills dir: ./runtime/skills
[inno] workspace: ./workspace
[inno] registering provider: innospark (anthropic-messages)
[inno] scheduler started, checking every 60s
[inno] server listening on http://localhost:3000
```

看到 `server listening on http://localhost:3000` 说明启动成功。

打开浏览器访问 `http://localhost:3000`，你会看到 Inno Agent 的 Web UI 界面。

【截图占位】 此处插入截图——浏览器打开 `http://localhost:3000` 后的 Web UI 初始界面，展示空的对话区和左侧会话侧栏

2.8 完成第一次对话

在底部输入框输入消息，按回车发送：

```
你好，帮我解释一下什么是机器学习中的过拟合
```

如果 Agent 正常流式输出回复，说明：API Key 配置正确、网络可访问服务商 API、前后端通信正常。

验证 L1 记忆是否工作：回复完成后，点击右侧「学习档案」标签页，应该能看到 Agent 自动记录了一条 `concept_explained` 类型的学习事件。如果看到了，说明三层记忆系统也在正常工作。

【截图占位】 此处插入截图——左侧对话区显示 Agent 的第一条回复，右侧学习档案面板展示刚产生的 `conceptexplained` 学习事件_

2.9 启动 CLI 模式

如果不需要 Web UI，或在无图形界面的服务器上使用：

```
npm run start -- --home ./runtime --workspace ./workspace
```

终端会显示 TUI 界面，在提示符处直接输入消息、回车发送即可对话。CLI 和 Web UI 共享同一个 `runtime/` 目录，CLI 里对话积累的学习记录，在 Web UI 里同样可见。

【截图占位】 此处插入截图——CLI TUI 界面，展示命令行对话提示符和 Agent 的一条回复

2.10 开发模式（前后端分离，适合调试代码时使用）

需要修改前端代码并看到热更新效果时，推荐两终端开发模式：

```
# 终端 1：启动后端（监听 :3000）
npm run dev:server

# 终端 2：启动前端 Vite 开发服务器（监听 :5173，自动代理 /api → :3000）
npm run web:dev
```

浏览器访问 `http://localhost:5173`（注意不是 3000）。

各类改动的重启规则：

改动内容	需要做什么
修改了 <code>src/</code> 下的后端 TypeScript	停止后端 → <code>npm run build</code> → 重启后端
修改了 <code>web/src/</code> 下的组件或样式	无需操作，Vite HMR 自动热更新，刷新页面即可
修改了 <code>web/vite.config.ts</code>	停止前端 → 重启 <code>npm run web:dev</code>
Wiki / 定时任务 / 渠道行为异常	优先完整重启前后端
修改了 <code>config.json</code> 配置	重启后端（启动时读取，模型切换是唯一热写例外）

2.11 健康检查

服务运行后，用以下命令快速确认各子系统状态：

```
# 后端服务是否存活
curl http://localhost:3000/health
# 正常返回: {"status":"ok","timestamp":"2026-06-05T..."}

# Wiki 模块是否正常
curl http://localhost:3000/api/wiki/pages
# 正常返回: [] 或 JSON 数组（首次启动为空数组）

# 定时任务调度器是否正常
curl http://localhost:3000/api/jobs/status
# 正常返回: 包含 enabled、jobCount 等字段的 JSON 对象
```

正常启动判断标准：

观察到的现象	判定
浏览器打开 <code>localhost:3000</code> 显示 UI 界面	前端构建正常，HTTP 服务启动成功
发送消息后 Agent 有流式回复	API 配置正确，模型可连接
回复完成后「学习档案」出现学习事件	L1 记忆系统工作正常
<code>curl /health</code> 返回 <code>{"status":"ok"}</code>	后端进程健康
浏览器打开显示空白页或 Cannot GET /	前端未构建，需要重新 <code>npm run build</code>

观察到的现象	判定
发送消息后加载圈一直转，无回复	API 配置有问题，检查 apiKey/baseUrl/api 字段
终端有警告日志但 UI 能正常使用	通常是可选模块问题，不影响核心功能

2.12 常见问题排查

Q1: `npm install` 中途报错，提示网络超时或 `ECONNRESET` ?

国内网络访问 npm 官方源可能不稳定：

```
npm install --registry https://registry.npmmirror.com
```

如果仍然失败，清除缓存后重试：

```
npm cache clean --force
npm install --registry https://registry.npmmirror.com
```

Q2: `npm run build` 报 TypeScript 编译错误?

最常见原因是 Node.js 版本不满足要求：

```
node --version # 必须 >= v20.6.0
```

同时确认在项目根目录执行命令（不是 `apps/inn-agent/` 子目录）：

```
pwd # 应该显示 ../inn-agent（仓库根目录，而不是子目录）
```

Q3: 启动后访问 `localhost:3000` 显示空白页或 `Cannot GET /` ?

前端未构建，`apps/inn-agent/web/dist/` 目录不存在或为空：

```
npm run build
npm run server -- --home ./runtime --workspace ./workspace
```

Q4: 发送消息后一直在加载，没有 Agent 回复?

这是 API 配置问题，按以下顺序排查：

1. 浏览器开发者工具 (F12) → Network 标签 → 找到 `/api/chat/stream` 请求 → 查看响应内容, 找到具体报错

2. 检查 `config.json`:

- `apiKey` 是否已填写 (不是模板里的占位文字)
- `baseUrl` 是否正确 (末尾不要加 `/v1` 等路径)
- `api` 字段是否与服务商一致 (见 2.6 节速查表)

3. 确认 API Key 有可用余额 / 额度未耗尽

Q5: 报错 `Provider not found` 或 `Model not found` ?

`defaultProvider` 的值必须和 `providers` 对象里某个 key 完全一致 (大小写敏感):

```
// ❌ 错误: 大小写不匹配
"defaultProvider": "InnoSpark",
"providers": { "innospark": { ... } }

// ✅ 正确
"defaultProvider": "innospark",
"providers": { "innospark": { ... } }
```

Q6: 端口 3000 被占用, 提示 `EADDRINUSE` ?

```
# 方式 A: 修改 config.json
"server": { "port": 8080 }

# 方式 B: 启动参数覆盖 (优先级高于配置文件)
npm run server -- --home ./runtime --workspace ./workspace --port 8080
```

然后访问 `http://localhost:8080`。

Q7: 使用本地 Ollama, Agent 不调用工具, 只输出文字?

本地小模型的 function calling 能力通常弱于云端大模型:

1. 确认使用支持 function calling 的模型 (推荐 `qwen2.5:14b` 及以上)
2. 如工具调用仍不稳定, 换更大参数量的模型, 或改用云端 API

Q8: macOS 上 Electron 版提示「此应用已损坏, 无法打开」?

这是 macOS Gatekeeper 对未签名应用的限制, 仅 Electron 打包版会出现, `npm run server` 方式启动不会有此问题。Electron 版解决方式参见 `ELECTRON_BUILD.md`。

Q9: Windows PowerShell 执行 npm 命令提示「不是内部或外部命令」？

Node.js 安装时未正确添加到系统 PATH:

1. 「开始菜单」→ 搜索「环境变量」→「编辑系统环境变量」→「环境变量」
2. 在「系统变量」中找到 `Path`，双击编辑
3. 确认列表中有 Node.js 安装路径（通常是 `C:\Program Files\nodejs\`）
4. 点击确定后，关闭并重新打开 PowerShell / CMD

Q10: 配置了多个 Provider，想在运行中切换？

两种方式均无需重启服务：

- **Web UI**：点击右侧「设置」标签页 → 在模型列表中直接点击选择，立即生效
- **手动改配置**：修改 `config.json` 的 `defaultProvider` 和 `defaultModel`，然后重启服务

Q11: 配置修改后保存了，但重启后没有生效？

确认修改的是 `runtime/config/config.json`，而不是仓库根目录的 `config.example.json`（这是模板文件，不是实际使用的配置）。

小结

- Node.js 版本必须 $\geq 20.6.0$ ；`npm install` 后必须执行 `npm run build`；两步缺一不可
- 配置只需改一个文件：`runtime/config/config.json`，核心是填对 `apiKey`、`baseUrl`、`api` 三个字段
- `api` 字段填错是最常见错误：Anthropic 系填 `anthropic-messages`，OpenAI 系填 `openai-completions`
- 启动成功的标志：终端出现 `server listening`，浏览器 UI 加载完成，发消息有回复，学习档案出现事件记录
- 下一章我们来详细了解 Web UI 的各个功能区域

第 3 章 Web UI 基础使用

学完本章，你将熟悉 Web UI 的每个功能区域，能够流畅地管理会话、发送多模态消息、切换模型，并理解各区域之间的联动关系。

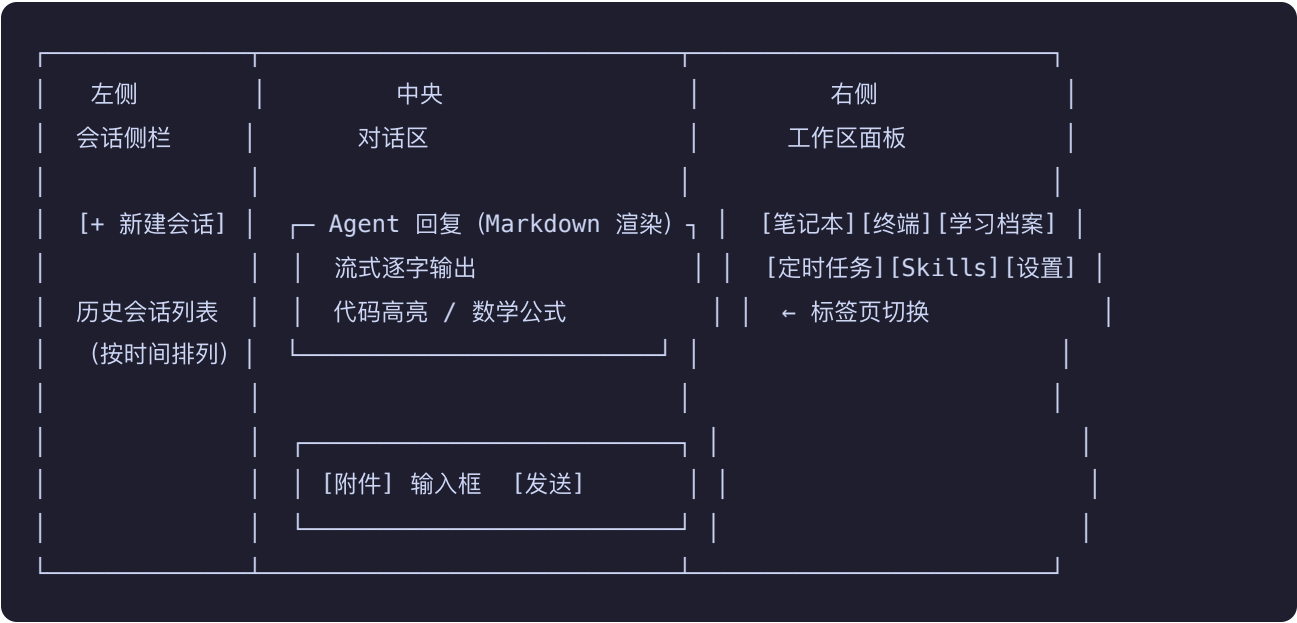
学习目标

1. 掌握三栏布局的结构，能准确定位每个功能区域
2. 熟练进行会话的创建、切换、工作区绑定和删除

- 3. 学会发送文字、图片和文件消息
- 4. 理解对话区消息格式的渲染规则
- 5. 掌握实时模型切换和工作区文件浏览

3.1 界面总览

打开 `http://localhost:3000`，你看到的是一个三栏学习工作台：



三栏的职责分工：

区域	内容	是否跨会话共享
左侧会话侧栏	会话列表、新建入口	—
中央对话区	消息流、输入框	L3 历史不共享（每个会话独立）
右侧工作区面板	笔记本/档案/设置等标签	L1/L2 全局共享，所有会话可见

【截图占位】 此处插入截图——Web UI 完整三栏界面全貌，用彩色框分别标注三个区域及右侧各标签名称

右侧工作区面板标签页说明：

标签	主要功能
笔记本	L2 知识 Wiki 页面列表 + 知识图谱可视化，上传文档入口

标签	主要功能
终端	内嵌 xterm.js 终端 (Practice Lab)，代码运行环境
学习档案	L1 学习者画像：目标、知识状态、误解、偏好，均可编辑
定时任务	Cron 任务列表，创建/编辑/手动触发/查看运行历史
Skills	安装、启用/禁用、删除 Skill 扩展
工作区	当前会话绑定的工作区文件树，支持预览和在线编辑
设置	Provider 和模型选择，渠道配置（飞书等）

3.2 左侧会话侧栏详解

左侧栏是所有会话的管理入口。

3.2.1 新建会话

点击顶部的「+ 新建会话」按钮，弹出新建对话框。

你可以： – 直接点击「创建」：使用默认配置，工作区目录为启动时指定的 `--workspace` 目录 – 选择「绑定工作区」：从已有工作区列表选择一个绑定（绑定后 Agent 的文件操作默认在该目录进行）

【截图占位】 此处插入截图——点击「+ 新建会话」后弹出的新建对话框，展示工作区选择下拉列表

新建完成后，左侧栏会出现一个新的会话条目，中央对话区清空，可以开始新的对话。

3.2.2 切换会话

直接点击左侧栏中任意历史会话条目即可切换。切换时：

- 中央对话区：加载该会话的 L3 历史（对话消息列表）
- 右侧工作区：保持当前状态（L1/L2 全局共享，不随会话切换而变化）

如果一个会话绑定了特定工作区，右侧「工作区」标签页会显示对应目录的文件树。

【截图占位】 此处插入截图——左侧会话侧栏，展示多个历史会话条目，当前激活的会话高亮显示

3.2.3 删除会话

将鼠标悬停在会话条目上，右侧会出现删除图标（垃圾桶），点击后二次确认即删除。

注意：删除会话会永久清除该会话的 L3 历史（对话记录），但 L1 和 L2 数据不受影响。

3.3 中央对话区详解

中央对话区是和 Agent 交互的主舞台。

3.3.1 消息格式与渲染

Agent 的回复支持完整的 Markdown 渲染：

内容类型	渲染效果
标题、加粗、列表	标准 Markdown 格式
代码块 (<code>python ...</code>)	语法高亮，支持 Python / JS / TypeScript / Go / Rust / SQL / CSS 等
数学公式 (<code>\$...\$</code> 或 <code>\$\$...\$\$</code>)	KaTeX 渲染，行内和块级均支持
表格	GFM 表格格式渲染
工具调用	以折叠块形式展示 (「Thinking & tool calls」)，可展开查看

【截图占位】 此处插入截图——对话区展示一条含代码块和数学公式的 Agent 回复，展示 Markdown 渲染效果

3.3.2 发送文字消息

在底部输入框输入文字，按 **Enter** 键或点击右侧发送按钮。

按 **Shift + Enter** 换行（不发送）。

Agent 开始生成回复时，会先出现「Thinking & tool calls」折叠区域（如果 Agent 调用了工具），然后正文内容流式逐字输出。

3.3.3 发送图片

两种方式：

- **点击附件图标：** 点击输入框左侧的附件按钮 (📎)，选择图片文件
- **粘贴：** 在输入框聚焦状态下，直接 `Cmd/Ctrl + V` 粘贴剪贴板中的图片

图片自动转为 base64 随消息发送给模型。支持多模态能力的模型（如 Claude Sonnet 系列）可以理解图片内容。

提示：如果你截图了一道数学题或代码报错截图，直接粘贴给 Agent 是最快的提问方式。

【截图占位】 此处插入截图——输入框中已粘贴一张图片的状态，展示图片缩略图预览

3.3.4 常用对话指令

指令	说明
<code>/new</code>	在当前渠道开启一个新会话（清空 L3，从零开始）

在对话中，你也可以用自然语言让 Agent 操作记忆和知识库：

帮我把这段笔记归档到知识库

帮我更新我的学习目标：我想在两个月内学完 PyTorch 基础

帮我看我今天的学习状态怎么样

帮我创建一个每天晚上 10 点发送学习总结的定时任务

3.4 模型切换

点击右侧「设置」标签页，可以看到当前使用的 Provider 和模型，以及所有可切换的选项。

切换步骤：

- 1. 点击右侧「设置」标签页
- 2. 在「当前模型」下拉列表或模型列表中，点击目标模型名称
- 3. 系统提示「已切换到 XXX」，配置立即热写入 `config.json`，无需重启服务

【截图占位】 此处插入截图——设置面板中的模型切换界面，展示可选模型列表，当前模型高亮

不同模型的适用场景建议：

场景	推荐模型类型
日常学习问答、概念解释	Sonnet 系列（速度快、成本低）
复杂推理、数学证明	Opus 系列（能力强，较慢）
高频短回答、复习测验	本地小模型（低延迟，完全离线）

切换模型后，当前会话的后续对话立即使用新模型；历史消息不受影响。

3.5 工作区浏览与文件编辑

每个会话可以绑定一个**工作区目录** (Workspace)。绑定后，Agent 在该会话中的文件操作（读、写、执行代码）都默认在这个目录下进行。

点击右侧「工作区」区域（或 Practice Lab 运行代码后自动聚焦），可以看到文件树浏览器：

文件树操作：

- **浏览文件：** 点击文件夹展开/折叠，点击文件在右侧预览
- **预览支持格式：** Markdown（渲染预览）、代码文件（语法高亮）、图片、PDF
- **在线编辑：** 点击文件后，右侧出现编辑器（CodeMirror 驱动），支持多种语言的语法高亮，修改后点击「保存」写回磁盘

【截图占位】 此处插入截图——右侧工作区面板，展示文件树（左）和 Markdown 文件预览（右），侧边有「编辑」按钮

工作区切换：

如果你有多个项目，可以为不同会话绑定不同的工作区目录。在创建新会话时可以选择工作区，也可以在会话详情中修改绑定。

3.6 查看 Agent 的思考过程

Agent 每次回复时，如果调用了工具或进行了推理，会在消息顶部显示「Thinking & tool calls」折叠区域（通常是蓝色/灰色背景的折叠块）。

展开它，你可以看到：

- **工具调用记录：** Agent 调用了哪个工具、传入了什么参数、工具返回了什么
- **推理过程**（如果模型支持 extended thinking）：模型的内部思考链

这对于理解 Agent 的行为、判断记忆是否正确更新非常有用。

【截图占位】 此处插入截图——对话区一条消息展开「Thinking & tool calls」折叠块，展示 recordlearningevent 工具调用的参数和结果

3.7 实战示例：第一周的典型使用流程

以下是一个初学者在第一周使用 Inno Agent 的典型流程，帮助你对各功能建立完整感知：

第 1 天：声明学习目标

我想在一个月内学完机器学习基础，包括线性回归、决策树、神经网络，目标是能独立完成 Kaggle 入门竞赛

Agent 会调用 `record_learning_event` (`goal_declared` 类型) 记录这个目标到 L1 画像。你可以在「学习档案」里看到刚创建的目标条目。

第 2 天：学习新概念

帮我解释梯度下降是什么，用一个生活中的例子

Agent 解释完后，掌握度自动更新 (`gradient_descent` 概念 +0.02)。

第 3 天：归档学习材料

上传一份 PDF 讲义 → 自动摘要并进入 L2 Wiki → 之后随时可以问「你对这份讲义有什么印象？」

第 4 天：做练习题

给我出一道关于线性回归的编程题，中等难度

Agent 出题，你作答，Agent 评判 → `exercise_attempt` 事件记录 → 掌握度更新。

第 7 天：查看学习进度

打开「学习档案」→ 查看本周涉及的所有概念的掌握度变化 → 或问：「总结一下我这周的学习情况」

3.8 常见问题

Q1：发送消息后中央区域一直显示加载状态，没有输出？

参见第 2 章 Q4 (API 配置问题)。最快的排查方式：打开浏览器 F12 开发者工具 → Network → 找 `/api/chat/stream` 请求 → 查看响应是否有错误。

Q2：Agent 的回复里数学公式显示为原始 LaTeX 字符串，没有渲染？

数学公式渲染由前端的 KaTeX 处理。确认公式使用了正确格式：行内公式 `$...$`，块级公式 `$$...$$`。如果格式正确但仍不渲染，尝试硬刷新页面 (`Shift+F5`)。

Q3：粘贴图片后发送，Agent 说「没有看到图片」？

确认使用的模型支持多模态输入 (Claude Sonnet/Opus 系列支持；部分本地模型不支持)。如果使用的是本地 Ollama 模型，需要确认该模型有视觉能力 (如 llava 系列)。

Q4：历史会话太多，想快速找到某个对话？

左侧会话列表按时间倒序排列，目前不支持全文搜索。临时解决方案：直接问 Agent 「我们之前是否讨论过 XXX 话题？」——如果在 L3 历史窗口内，Agent 可以回答。

Q5：切换会话后，右侧「学习档案」里的数据消失了？

不会消失。L1 画像是全局共享的，切换会话后你看到的仍然是同一份画像。如果看起来「空了」，可能是切换后右侧面板还在加载中，稍等片刻即可。

小结

- 三栏布局：左会话管理 / 中对话 / 右多功能工作区，右侧 L1+L2 全局共享
- 发消息支持文字、图片（粘贴或上传）、`/new` 指令
- Agent 回复支持 Markdown 全渲染（代码高亮、数学公式、表格），工具调用可展开查看
- 模型切换无需重启，热写配置文件
- 下一章我们深入了解三层记忆系统，学会主动查看和修正 Agent 对你的认知

第 4 章 分层记忆系统

学完本章，你将深入理解三层记忆的数据结构和更新机制，能够主动查看、修正 Agent 对你的认知，并判断什么信息该写入哪层记忆。

学习目标

1. 理解 L1 画像中四个核心字段的含义和写入规则
2. 掌握全部 6 种学习事件类型及其对掌握度的影响
3. 了解误解（Misconception）的完整生命周期
4. 能够通过 Web UI 查看、手动修正、乃至重建学习者画像
5. 理解 L2/L3 各自的职责边界

4.1 L1：学习者画像全貌

L1 是系统对「你作为学习者」的持久认知，存储在 `data/learner/profile.json` 中，包含四个核心字段：

① goals（学习目标）

每个目标包含：

```
interface LearningGoal {
  goal_id: string;
  title: string;
  type: "skill" | "concept" | "project" | "exam" | "habit";
  priority: number;          // 0-1, 越高越优先出现在上下文包里
  status: "active" | "paused" | "completed" | "archived";
  success_criteria: string[];
  source: "user_declared" | "agent_inferred" | "imported";
  updated_at: string;
}
```

② knowledge_states (概念掌握度)

每个概念记录精细的学习状态:

```
interface KnowledgeState {
  concept_id: string;
  concept_name: string;
  domain: string;
  mastery: number;          // 0-1, 当前掌握度
  confidence: number;       // 0-1, 系统对这个估计的信心
  stability: number;        // 0-1, 掌握度是否稳定
  last_practiced_at?: string;
  review_due_at?: string;   // 下次复习时间, 到期后进入上下文包
  evidence_ids: string[];   // 对应的学习事件 ID 列表
  diagnosis: string;        // 当前薄弱点诊断
  next_actions: string[];   // 建议的下一步学习动作
}
```

③ misconceptions (已识别误解)

```
interface Misconception {
  misc_id: string;
  description: string;
  severity: "low" | "medium" | "high";
  confidence: number;       // 系统对这个误解判断的信心
  status: "active" | "repairing" | "resolved" | "stale";
  first_seen_at: string;
  last_seen_at: string;
  evidence_ids: string[];
```

```
repair_strategy: string; // Agent 建议的修复方式
}
```

④ preferences（教学偏好）

四个字符串列表，影响 Agent 在每轮的教学策略：

```
interface LearnerPreferences {
  explanation_style: string[]; // 如 ["example_first", "visual_analogy"]
  practice_style: string[];    // 如 ["code_exercises", "small_steps"]
  feedback_tone: string[];     // 如 ["encouraging", "direct"]
  avoid: string[];             // 如 ["long_derivations", "abstract_first"]
}
```

【截图占位】 此处插入截图——右侧「学习档案」标签页总览，展示目标列表（左上）、知识状态列表（右上）和偏好设置（下方）

4.2 学习事件：L1 的写入单元

L1 的更新不直接修改 profile 字段，而是先追加学习事件到 events.jsonl，再由确定性规则从事件折叠出画像更新。这样做的好处是：事件日志是不可变的，任何时候都可以从事件重建画像；Agent 的每个推断都有证据可查。

全部 6 种事件类型：

事件类型	触发时机	对画像的影响
goal_declared	用户声明或调整学习目标	创建/更新目标，设置优先级和状态
concept_explained	Agent 完成一次概念讲解	对应概念掌握度 +0.02，稳定性轻微提升
exercise_attempt	做了一道练习题（不论对错）	对应概念掌握度 +0.03（幅度最大，因为主动练习效果最好）
self_assessed	用户主动评价自己的理解	掌握度 +0.01，confidence 更新
feedback_received	用户给出正面或负面反馈	调整相关概念的 confidence 和 next_actions
milestone_reached	完成了一个阶段性目标	相关概念掌握度 +0.02，目标状态可能变为 completed

掌握度更新规则细节：

- 每个事件 ID 在 `evidence_ids` 中只计一次，防止重放同一事件反复涨分
- `derived_signals.mastery_delta` 字段可以覆盖默认 delta（Agent 对困难的练习题可以设定更高的 delta）
- 掌握度值域为 `[0, 1]`，超出范围自动截断

复习时间排期规则：

当前掌握度	下次复习间隔
< 0.40（薄弱）	+1 天
0.40 — 0.75（掌握中）	+3 天
≥ 0.75（熟练）	+7 天

这是间隔重复（Spaced Repetition）的简化实现。当 `review_due_at` 到期，该概念自动出现在上下文包的 `review_due_concepts` 列表里，提示 Agent 在这轮对话中安排复习。

4.3 误解的生命周期

误解（Misconception）不是一个布尔值，而是有完整生命周期的状态机：

识别阶段：Agent 在对话中发现潜在误解

- 调用 `record_learning_event`，携带 `misconception_candidates` 信号
- 系统评估信心度，超过阈值后写入 `misconceptions` 字段
- 状态：active（活跃）

修复阶段：Agent 针对误解进行专项讲解或练习

- 状态：repairing（修复中）

解决阶段：Agent 确认误解已纠正（用户答对相关练习/自评已理解）

- 状态：resolved（已解决）
- 该误解不再出现在上下文包的活跃误解列表中

过期阶段：resolved 的误解经过较长时间没有新证据

- 状态：stale（过期）
- 可以安全清理

重要设计原则：系统采用保守策略——不会把每一个 `misconception_candidate` 信号自动晋升为正式误解，避免「把正在探索的问题误认为是错误认知」。

4.4 上下文包的精确内容

每轮对话的系统提示里，上下文包这一段大约包含以下信息：

```
=== 学习背景 ===
当前目标：[最高优先级 active 目标的 title]

相关薄弱概念（掌握度从低到高）：
  1. multi_head_attention 掌握度 0.23 诊断：混淆了 head 数量和 d_model 的关系
  2. positional_encoding 掌握度 0.41 诊断：理解位置编码的动机，但不清楚具体公式
  3. layer_normalization 掌握度 0.55 （无诊断）

活跃误解：
  - 误认为 Q/K/V 矩阵的输入必须来自不同序列（中等严重）

教学提示：
  - 用户偏好先给代码示例，再解释原理
  - 避免长篇数学推导

今日待复习：
  - scaled_dot_product_attention（复习到期）

近期学习事件（最近 8 条）：
  ...
```

这就是 Agent 每次回复你之前看到的「摘要」。它远小于完整的 `profile.json`，但包含了做出教学决策所需的全部关键信息。

4.5 L2：Wiki 知识记忆结构

L2 以原生 Wiki 的形式存储学习材料和知识，所有内容对人类可读可编辑。

四种页面类型：

类型	创建方式	说明
<code>source-summary</code>	自动（归档时生成）	归档文档的摘要，含标题、要点列表和 <code>[[概念]]</code> 链接
<code>entity</code>	自动 + 可手动编辑	描述人物、组织、项目、论文、库等实体
<code>concept</code>	自动 + 可手动编辑	描述技术概念、理论、方法、设计模式

类型	创建方式	说明
analysis	手动创建	你自己的分析：对比、学习路径、研究结论

磁盘目录结构：

```
data/l2/
├─ raw/                # 原始上传文件（只读，保留来源可追溯性）
├─ extracted/          # 提取的 Markdown 文本（只读）
├─ wiki/
│   ├─ index.md        # 每次归档后重建的全局索引（Agent 优先读这里）
│   ├─ log.md          # 操作追加日志（时间戳 + 操作描述）
│   ├─ sources/        # source-summary 页面
│   ├─ entities/       # entity 页面
│   ├─ concepts/       # concept 页面
│   └─ analysis/       # analysis 页面
└─ manifest.jsonl      # 追加式元数据索引（包含 SHA-256、路径、标签等）
```

Wiki 页面的 YAML frontmatter 格式：

每个 Wiki 页面顶部有一段 YAML 元数据，例如一个概念页：

```
---
title: Multi-Head Attention
type: concept
created: "2026-05-20"
updated: "2026-06-01"
tags: ["transformer", "attention", "deep-learning"]
sources: ["Attention Is All You Need 摘要"]
source_ids: ["l2src_abc123"]
status: reviewed
confidence: high
contested: false
---

## 概述
Multi-Head Attention 是 Transformer 的核心组件...

## 关键公式
...
```

```
## 与 [[Self-Attention]] 的关系

...
```

`confidence` 字段 (`low` / `medium` / `high`) 和 `status` (`draft` / `reviewed` / `outdated`) 帮助你 和 Agent 判断哪些页面内容可信、哪些需要更新。

4.6 L3：会话历史

L3 由 Pi SDK 的 `SessionManager` 全权管理，Inno Agent 不直接读写它。L3 的职责：

- **存储近期对话**：每轮用户消息 + Agent 回复 + 工具调用记录
- **自动压缩**：当对话历史接近模型的上下文限制时，SDK 自动将较早的轮次压缩为摘要
- **会话隔离**：每个 Web UI 会话、每个飞书聊天，各自有独立的 L3 空间

Inno Agent 在 L3 之上做了一件额外的事：通过 `WorkspaceRegistry` 将每个会话绑定到工作区目录，让 Agent 在每轮开始时能拿到当前工作区的 README 和最近的运行记录（Practice Lab 输出），加入系统提示。

4.7 在 Web UI 中查看与修正记忆

打开右侧「学习档案」标签页，你会看到三个子面板：

子面板 A：学习目标

列出所有 `active` 和 `paused` 状态的目标，按优先级排序。

可以执行的操作： – **新建目标**：点击「添加目标」，填写标题、类型、成功标准 – **修改优先级**：拖动目标条目或直接修改优先级数值 – **归档目标**：点击目标右侧「归档」，目标变为 `archived` 状态，不再出现在上下文包里 – **标记完成**：点击「完成」，状态变为 `completed`

【截图占位】 此处插入截图——学习目标子面板，展示多个目标条目，包含类型标签 (skill/concept) 和优先级数值

子面板 B：知识状态

列出所有被记录过的概念，按掌握度升序（最弱的在最前面）。

可以执行的操作： – **查看诊断**：点击某个概念，展开查看完整的诊断描述和历史证据事件 ID – **手动修改掌握度**：如果 Agent 的判断不准确，可以直接输入数值覆盖 – **修改下次复习时间**：调整 `review_due_at` 字段 – **清除某个概念**：如果某个概念被错误记录，可以删除它

【截图占位】 此处插入截图——知识状态子面板，点击一个概念后展开的详情区，展示掌握度滑块、诊断文本框和证据事件列表

子面板 C：误解与偏好

- **误解列表：**查看所有 `active` 状态的误解，可以手动将某条标记为「已解决」（如果你确认自己已经理解正确了）
- **偏好设置：**修改解释风格、练习偏好、反馈语气、回避项；修改后立即影响下次对话中的上下文包

【截图占位】 此处插入截图——偏好设置区域，展示解释风格和练习风格的标签选择界面

4.8 从零重建画像

如果你觉得 Agent 对你的认知已经完全偏了，有两种重置方式：

方式 A：清空并重建（彻底）

```
# 停止服务后执行
rm data/learner/profile.json
rm data/learner/events.jsonl
```

重启服务后，Agent 从空白画像开始，重新通过对话积累认知。

方式 B：从事件日志重建（保留历史）

如果 `profile.json` 的汇总出现了问题，但 `events.jsonl` 的原始记录是准确的：

在对话中输入：

请根据我的学习事件日志，从头重新生成我的学习者画像

Agent 会调用内部的 `rebuildProfileFromEvents` 函数，重放所有事件重新计算画像，清理累积的误差。

4.9 常见问题

Q1: Agent 说「你已经掌握了 X」，但我明明还没学过？

可能是 Agent 在对话中过于乐观地记录了 `concept_explained` 事件。两个解决方式：

1. 直接告诉 Agent：「我对 X 的掌握度标注不对，请帮我降低到 0.1」—— Agent 会调用 `patch_learner_profile` 修改

2. 在「学习档案」→「知识状态」子面板中手动修改掌握度数值

Q2: 学习档案里没有我期望的概念?

Agent 只记录它在对话中明确讲解或练习过的概念。如果你想主动注册一个概念，可以告诉 Agent:「帮我在学习档案里添加 '矩阵乘法' 这个概念，掌握度先设为 0.2」。

Q3: 偏好设置改了，但 Agent 的回答风格没变?

偏好修改会在下一轮对话的上下文包中生效（不是当前这轮）。如果改了但下轮仍然没效果，检查偏好字段是否成功保存（刷新页面后检查是否还显示新值）。

Q4: `events.jsonl` 文件会越来越大吗？有什么清理机制？

会缓慢增长，但单条事件很小（< 1 KB），1000 条事件通常不超过 1 MB。目前没有自动清理机制。如果文件变得很大（> 10 MB），可以在服务停止后手动归档旧事件：

```
# 只保留最近 500 条事件（旧事件已被折叠进 profile.json，可以安全归档）
tail -n 500 data/learner/events.jsonl > events_recent.jsonl
mv events_recent.jsonl data/learner/events.jsonl
```

Q5: L1 和 L2 的信息会互相影响吗？

L1 存的是「你对知识的掌握状态」（主观），L2 存的是「知识本身的内容」（客观）。Agent 在回答时可以同时查询两者：先从 L2 里找到关于某个概念的权威说明，再根据 L1 里你对这个概念的掌握度来调整解释深度。两者之间的链接通过 `concept_id` 和 `concept_name` 对齐——L1 里的 `concept_id` 和 L2 中 `concept` 类型页面的标题在语义上对应。

小结

- L1 = 关于「你」的知识（目标、掌握度、误解、偏好），每轮自动精简注入
- L2 = 关于「知识本身」的内容（文档摘要、概念页、实体页），按需检索
- L3 = 近期对话流水，由 Pi SDK 负责，Inno 层不直接干预
- 学习事件是 L1 的写入单元，6 种类型覆盖主要学习场景；掌握度有防重放机制
- 误解有完整生命周期，从识别到解决全程可追踪
- 所有 L1 数据都可以通过 Web UI「学习档案」面板直接查看和修改

第 5 章 知识 Wiki 进阶操作

学完本章，你将能熟练归档各类学习材料，利用知识图谱发现概念间的关系，并掌握手动维护 Wiki 的完整操作流程。

学习目标

- 1. 理解文档摄入流程的每个环节，能判断归档结果是否正确
- 2. 掌握通过对话和 Web UI 两种方式查询知识库
- 3. 学会在知识图谱中导航、理解节点和边的含义
- 4. 能手动创建 analysis 页面，管理 Wiki 内容质量

5.1 归档文档：支持格式与注意事项

支持的文件格式：

格式	说明
PDF	最常用，支持文字 PDF（含学术论文）；扫描版 PDF（纯图片）需要 OCR，当前版本不支持
DOCX	Word 文档，正文文字提取，图片暂时忽略
PPTX	PowerPoint，提取每页文字内容
XLSX	Excel，提取工作表中的文字单元格
PNG / JPG / GIF 等	图片文件，直接发给多模态模型提取文字描述
Markdown 文本	直接粘贴到对话，Agent 归档纯文本内容

单文件最大 100 MB。文件经由 @llamaindex/liteparse 解析，转换为 Markdown 后归档。

5.2 归档操作步骤

方式 A：通过 Web UI 上传（推荐）

- 1. 点击右侧「笔记本」标签页
- 2. 找到上传区域（页面顶部或侧边栏的「上传文档」按钮）
- 3. 拖入文件，或点击按钮选择文件
- 4. 进度条完成后，页面列表会出现新增的 source-summary 页面

【截图占位】 此处插入截图——笔记本面板，展示文件上传拖拽区域（高亮蓝色边框状态）和上传进度条

方式 B：通过对话指令

如果文件已经在工作区目录里（例如你放在 `workspace/papers/` 下的 PDF）：

帮我把工作区里 `papers/attention_is_all_you_need.pdf` 归档到知识库

如果想直接归档一段文字（如网页内容、课堂笔记）：

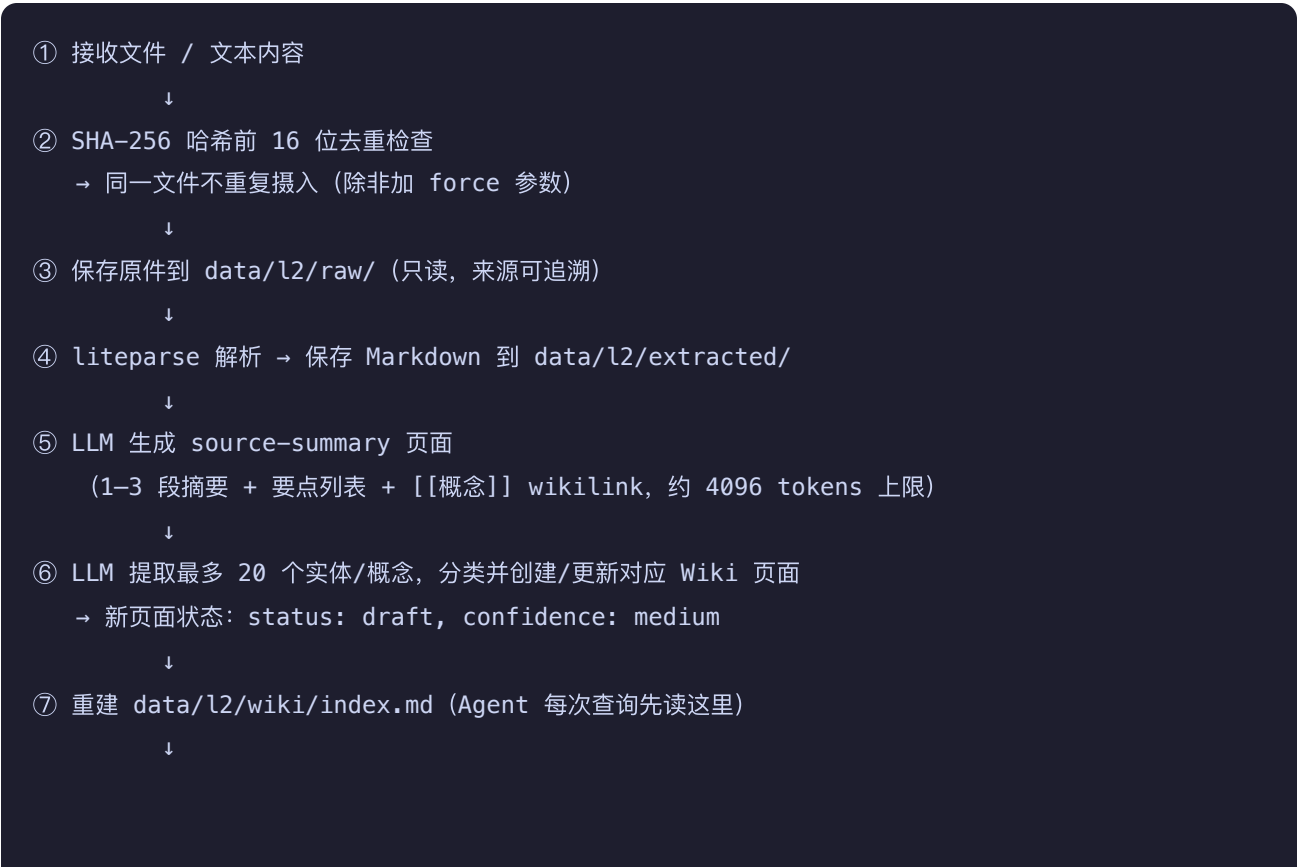
帮我把以下内容归档到知识库，标题是「梯度下降笔记」：

[粘贴笔记内容]

Agent 会调用 `l2_archive` 工具，流程与 Web UI 上传完全一致。

5.3 摄入流程详解

每次归档都经历以下固定流程，全程自动完成：



- ⑧ 追加记录到 manifest.jsonl (含 SHA-256、路径、标签、状态)
- ⑨ 追加操作记录到 wiki/log.md

为什么 raw/ 和 extracted/ 是只读的?

系统提示词明确禁止 Agent 修改这两个目录，原因是：Wiki 页面的 `sources` 字段记录了它的来源文件路径；一旦原件被改动，来源可追溯性就断了，将来无法验证某个说法是否真的来自那份文献。

5.4 查询知识库

通过对话（最常用）

Transformer 的 multi-head attention 和 self-attention 有什么区别？
你的知识库里面有没有关于这个的资料？

Agent 调用 `l2_query` 工具，执行两阶段查询：

1. 索引阶段：读取 `wiki/index.md`，获取所有页面的标题、类型、标签概览
2. 内容阶段：在 manifest 中按关键词匹配，找到最相关的 5 个页面，读取全文内容

返回的回答中，对知识库内容的引用会用 `[[页面名]]` 标注，你可以追问具体某个页面的内容。

通过 Web UI 搜索

在「笔记本」标签页顶部搜索框输入关键词：

- 搜索范围：页面标题、tags 字段、页面正文全文
- 支持中英文混搜
- 结果按相关度排序，点击结果条目直接进入该页面

【截图占位】 此处插入截图——笔记本标签页，搜索框输入「attention」后出现搜索结果列表，高亮匹配词

5.5 知识图谱可视化

打开图谱

点击「笔记本」标签页顶部的「知识图谱」切换按钮（或图标），页面从列表视图切换到图谱视图。

图谱元素说明：

元素	含义	视觉样式
节点（大圆）	<code>source-summary</code> 类型页面	橙色
节点（中圆）	<code>entity</code> 类型页面	蓝色
节点（小圆）	<code>concept</code> 类型页面	绿色
节点（菱形）	<code>analysis</code> 类型页面	紫色
边（实线）	页面内 <code>[[wikilink]]</code> 引用	灰色箭头

操作说明：

- **缩放：**鼠标滚轮放大/缩小
- **平移：**拖动画布空白区域
- **选中节点：**单击节点，右侧出现该页面的详情卡片（标题、类型、标签、摘要、来源）
- **进入页面：**双击节点，跳转到该页面的编辑/预览视图
- **图谱布局：**支持多种自动布局算法（cola、cose-bilkent），可在图谱工具栏切换

【截图占位】 此处插入截图——知识图谱视图全貌，多个颜色节点和连线，右侧展开一个节点的详情卡片

图谱的实际用途：

- **发现未知联系：**你归档了 5 篇论文，图谱里会出现它们共同引用的概念节点，这些节点就是交叉知识
- **识别孤立知识：**没有连线的孤立节点说明这个概念还没有被其他页面引用，可能需要人工建立联系
- **追踪来源：**从概念节点出发，沿箭头找到引用它的 `source-summary`，知道这个概念来自哪篇文献

5.6 在 Web UI 中编辑 Wiki 页面

在「笔记本」页面列表中点击任意页面，进入编辑视图（分为预览模式和编辑模式，右上角切换）。

编辑视图的两个区域：

Frontmatter 区域（上方）：显示 YAML 元数据，可以修改的字段包括：

字段	说明
<code>title</code>	页面标题

字段	说明
<code>tags</code>	标签列表（影响搜索和图谱聚类）
<code>status</code>	<code>draft</code> / <code>reviewed</code> / <code>outdated</code>
<code>confidence</code>	<code>low</code> / <code>medium</code> / <code>high</code> （这条内容的可信度）
<code>contested</code>	<code>true/false</code> （内容是否有争议）

Markdown 内容区域（下方）：CodeMirror 驱动的编辑器，支持：

- 语法高亮（Markdown 内联预览）
- `[[wikilink]]` 语法（在图谱中生成边）
- 保存快捷键： `Cmd/Ctrl + S`

修改后点击「保存」按钮，文件即时写回 `data/l2/wiki/`。

【截图占位】 此处插入截图——Wiki 页面编辑视图，上方 *frontmatter* 区域展示字段，下方 *Markdown* 编辑器有语法高亮

5.7 手动创建 analysis 页面

`analysis` 类型页面是专门给你存放自己分析成果的地方——比如「对比两篇论文的方法论」「我的 PyTorch 学习路径规划」「这周读书的核心收获」。

创建步骤：

1. 在「笔记本」标签页，点击「新建页面」按钮
2. 选择类型： `analysis`
3. 填写标题和初始内容
4. 在正文中用 `[[概念名]]` 链接到相关概念或来源页面
5. 保存

建议内容结构：

```
---
title: Transformer 与 RNN 对比分析
type: analysis
tags: ["transformer", "rnn", "comparison"]
status: draft
confidence: medium
```

```
---
## 对比维度
```

```
...
```

```
## 关键差异
```

```
...
```

```
## 适用场景
```

```
...
```

```
## 参考来源
```

```
- [[Attention Is All You Need 摘要]]
```

```
- [[LSTM 基础]]
```

5.8 常见问题

Q1: 归档后页面列表没有新增条目?

最常见原因: 摄入流程在 LLM 生成摘要步骤失败 (API 超时或网络问题)。可以检查:

1. 查看 `data/l2/wiki/log.md`, 找最近一条记录, 看是否有 error 信息
2. 在对话中输入「检查最近一次归档是否成功」, Agent 会查询 `manifest.jsonl`

Q2: 归档后找不到新页面, 但 raw/ 目录有文件?

说明前几步 (保存原件、解析) 成功, 但 LLM 步骤 (生成摘要/提取概念) 失败。在对话中输入「更新知识库索引并重新生成最近归档文件的摘要」, 让 Agent 补全这些步骤。

Q3: 知识图谱节点太多, 看不清楚?

可以通过搜索筛选只显示相关节点: 在图谱工具栏的过滤框输入关键词, 只高亮匹配的节点。也可以按类型过滤 (只看 concept 节点, 隐藏 source-summary 节点)。

Q4: 上传同一份文件, 但想强制重新生成摘要怎么做?

在对话中明确指定 `force` 参数:

```
帮我重新归档工作区里的 paper.pdf, 忽略去重检查, 强制重新生成摘要
```

Agent 会在调用 `l2_archive` 时设置 `force: true`。

Q5: 某个概念页的内容是 AI 自动生成的, 有错误, 修改后会被覆盖吗?

不会。自动生成的页面存在 `wiki/` 目录下，属于可编辑区。你手动修改保存后，除非下次触发 `graphify_update` 时 Agent 认为需要更新该页面（通常只会在新归档文档后才会），否则不会覆盖你的修改。

建议修改后将 `status` 改为 `reviewed`，提示 Agent 这个页面已经过人工核对，减少自动覆盖的可能性。

小结

- 支持 PDF/DOCX/PPTX/图片/文本，最大 100 MB，SHA-256 自动去重
- 摄入 9 步全自动：原件 → 提取 → 摘要 → 概念链接 → 重建索引
- `raw/` 和 `extracted/` 只读，来源可追溯；`wiki/` 目录可以自由编辑
- 知识图谱是 `[[wikilink]]` 的可视化，能发现跨文献的概念关联
- `analysis` 页面是你自己的分析笔记，不受自动摄入覆盖

第 6 章 定时任务与主动推送

学完本章，你将能够设置定时任务，让 Agent 在无人值守时自动完成学习回顾、生成习题、推送报告，将被动等待变成主动学习循环。

学习目标

1. 理解定时任务的执行流程，以及它和普通对话的区别
2. 掌握 7 种内置任务类型的用途和适用场景
3. 在 Web UI 中创建、编辑、手动触发定时任务
4. 理解 Cron 表达式语法，能写出常用的时间规则
5. 配置任务结果推送到飞书（需先完成第 7 章配置）

6.1 定时任务的执行原理

定时任务不是「发给 Agent 的消息」，而是调度器在后台按时触发的**自动化工作流**。

执行流程：

```
调度器每 60 秒轮询一次 jobs.json
→ 检查哪些任务的 nextRunAt ≤ 当前时间
→ 找到到期任务，按序执行（同一时刻多个任务则依次运行）
```

↓

对于 `push_reminder` 类型：

- 直接格式化消息文本，推送到配置的渠道
- 不走 Agent 推理循环，速度快、成本低

对于其他类型 (`daily_review` 等)：

- 加载 L1 上下文包 (和普通对话一样)
- 通过 Agent 序列化运行器执行任务 Prompt
- 工具调用 (更新 L1/L2 等) 在执行过程中完成
- 输出结果格式化后 (如果配置了渠道) 推送

↓

运行结果追加到 `data/jobs/runs.jsonl`

- 包含：run_id、开始/结束时间、耗时、触发来源、输出摘要、推送状态
- 下次轮询时更新 `nextRunAt`

关键区别：定时任务用的是同一套三层记忆——即使你不在线，任务执行时 Agent 依然能读到你的最新画像，更新知识状态，并把更新写回 L1，供下次对话使用。

6.2 内置任务类型详解

`daily_review` (每日复习)

最常用的任务类型。执行逻辑：

1. 读取当天的学习事件列表 (`events.jsonl`)
2. 总结今天学了什么、练习了什么、有什么进展
3. 检查是否有 `review_due_at` 到期的概念，生成复习计划
4. 更新画像 (如果有明显的目标进度变化)
5. 输出「今日学习总结 + 明日建议」报告

推送到飞书或微信后，你在早晨查看手机就能看到昨晚的学习报告和今日复习建议。

`weekly_summary` (每周总结)

建议每周日晚上或周一早上运行。执行逻辑：

1. 汇总本周所有学习事件
2. 分析本周各概念掌握度的变化趋势
3. 评估目标完成进度
4. 生成下周学习建议

spaced_review (间隔复习)

自动为今天到期的概念生成复习习题。特点：

- 基于 L1 里每个概念的 `review_due_at` 判断是否到期
- 为到期概念生成 1—3 道习题，难度根据当前掌握度调整
- 发送习题后等待用户回答（需要通过飞书/Web UI 回复）
- 根据回答结果更新掌握度

最适合和飞书渠道配合使用——任务触发后习题推送到飞书，你在手机上随手作答。

learner_profile_reflection (画像反思)

建议每 3—7 天运行一次。执行逻辑：

1. 重新分析近期事件日志
 2. 检查是否有 `misconception_candidate` 信号应该晋升为正式误解
 3. 检查是否有目标应该调整优先级或归档
 4. 更新 `profile_summary` 字段，生成画像变化报告
-

graphify_update (知识库更新)

重新扫描 L2 中所有来源，重新提取实体和概念，更新 Wiki 图谱。适合在批量归档文档后运行，或者每周运行一次保持图谱新鲜。

push_reminder (推送提醒)

最简单的任务类型，直接推送一条固定文字消息，不经过 Agent 推理。常用场景：

- 每天早上 8 点：「记得今天要完成梯度下降的练习！」
 - 每周一：「本周学习目标提醒：[你的目标]」
 - 一次性提醒（设置 one-shot Cron，只触发一次）
-

custom_prompt (自定义 Prompt)

灵活度最高，可以让 Agent 执行任何你想要的定时任务。示例：

- 每天晚上 11 点，根据我今天的事件生成一首关于今日所学的四行打油诗，发给我
 - 每周五下午 6 点，汇总本周 Wiki 里新增的概念，生成一份「本周新知」卡片
 - 每月 1 日，用中文总结上个月的学习进度，发到我的飞书
-

6.3 在 Web UI 中创建定时任务

步骤：

- 1. 点击右侧「定时任务」标签页
- 2. 点击右上角「创建任务」按钮
- 3. 填写表单字段（见下方说明）
- 4. 点击「保存」

表单字段说明：

字段	必填	说明
任务名称	是	自定义名称，用于识别任务
任务类型	是	从 7 种内置类型中选择
Cron 表达式	是	触发时间规则，见 6.4 节
时区	是	强烈建议明确填写，如 <code>Asia/Shanghai</code>
推送渠道	否	<code>feishu</code> / <code>wechat</code> / 留空（只记录不推送）
推送目标	否	特定用户 ID；留空则推送到当前渠道默认目标
自定义 Prompt	仅 <code>custom_prompt</code>	填写要执行的具体指令

【截图占位】 此处插入截图——定时任务创建表单，展示所有字段及已填写的示例内容

通过对话创建（更快捷）

用自然语言描述，Agent 自动填写所有字段：

帮我设置：

- 每天晚上 9 点半，每日复习任务，结果推送到飞书
- 每周日下午 5 点，每周总结，推送到飞书
- 每天早上 8 点，推送一条「今天是新的开始，加油！」的提醒到飞书

Agent 调用 `create_scheduled_job` 三次，创建三个任务，并确认创建结果。

6.4 Cron 表达式速查

标准 5 字段格式：分 时 日 月 周

字段取值范围：

字段	范围	特殊值
分	0—59	* （每分）、 */5 （每 5 分）
时	0—23	* （每时）
日	1—31	* （每天）
月	1—12	* （每月）
周	0—6	0=周日，1=周一， 1-5 =工作日

常用表达式示例：

表达式	含义
0 22 * * *	每天 22:00
30 21 * * *	每天 21:30
0 8 * * 1	每周一 08:00
0 17 * * 5	每周五 17:00（周末前总结）
0 9 * * 1-5	工作日 09:00
0 8 * * 0	每周日 08:00（周报）
*/30 * * * *	每 30 分钟
0 0 1 * *	每月 1 日 0:00
30 14 15 * *	每月 15 日 14:30

一次性提醒（只触发一次，自动删除）

把日期固定为某一天，例如「2026 年 6 月 20 日 15:00 提醒我参加考试」：

0 15 20 6 *

Cron 调度器检测到日/月/分/时都精确指定（非 *），任务触发一次后自动禁用。

提示：不确定自己写的 Cron 对不对？直接把表达式发给 Agent：「这个 Cron 表达式 `0 9 * * 1-5` 是什么意思？」它会帮你解释。

6.5 查看和管理现有任务

在「定时任务」标签页，任务列表显示每个任务的：

- 任务名称和类型
- 下次运行时间（`nextRunAt`）
- 上次运行状态（成功/失败/跳过）
- 启用/禁用开关

操作：

- **手动立即运行**：点击任务右侧的「立即运行」按钮，等同于 Cron 触发（常用于测试任务是否正常工作）
- **编辑任务**：点击编辑图标，修改 Cron 表达式、类型或 Prompt
- **禁用任务**：关闭开关，任务暂停但不删除，`nextRunAt` 不再更新
- **删除任务**：点击删除图标，从 `jobs.json` 永久移除

【截图占位】 此处插入截图——定时任务列表，展示多个任务条目，每条显示状态标签（成功/失败）和下次运行时间

查看运行历史：

点击任务条目，展开「运行历史」列表，每条记录包含：

字段	说明
触发来源	<code>scheduled</code> （自动） / <code>manual</code> （手动） / <code>api</code> （接口调用）
开始时间	任务实际开始执行的时间
耗时	从开始到结束的毫秒数
状态	<code>success</code> / <code>error</code> / <code>skipped</code>
输出摘要	Agent 回复的前 500 字
推送状态	推送渠道返回的结果

【截图占位】 此处插入截图——任务详情展开后的运行历史列表，展示最近 5 次运行记录

6.6 实战示例：搭建完整的主动学习循环

以下是一套推荐的任务配置，覆盖「日回顾 + 间隔复习 + 周总结」：

任务 1：每日复习（晚上睡前）

```
名称：每日学习总结
类型：daily_review
Cron: 0 22 * * *
时区：Asia/Shanghai
渠道：feishu
```

任务 2：间隔复习习题（早上通勤）

```
名称：早间复习习题
类型：spaced_review
Cron: 30 7 * * *
时区：Asia/Shanghai
渠道：feishu
```

任务 3：每周总结（周日下午）

```
名称：周学习报告
类型：weekly_summary
Cron: 0 17 * * 0
时区：Asia/Shanghai
渠道：feishu
```

任务 4：知识库维护（每周自动更新图谱）

```
名称：Wiki 图谱更新
类型：graphify_update
Cron: 0 3 * * 0
时区：Asia/Shanghai
渠道：（不配置推送，静默运行）
```

这套配置的效果：你每天早上刷飞书看到今日复习题，晚上看到今日总结，周日下午收到本周报告——整个学习循环在你不打开电脑的时候也在自动运转。

【截图占位】 此处插入截图——飞书消息界面，展示一条 *dailyreview* 任务推送的学习总结消息（含概念列表、事件数量、明日建议） _

6.7 常见问题

Q1：定时任务到了时间但没有触发？

排查清单：

1. 确认任务的 `enabled` 开关是否打开（在任务列表中检查）
2. 确认 Cron 表达式是否正确（用「立即运行」测试）
3. 确认时区设置是否正确（`Asia/Shanghai` 表示北京时间）
4. 调度器每 60 秒轮询一次，有最多 60 秒延迟；不要在整点触发后立刻期望看到结果
5. 查看 `data/jobs/runs.jsonl` 最近几行，看是否有 error 记录

Q2：任务运行报错「No model configured」？

任务运行时使用的是当前 `config.json` 里的 `defaultProvider` 和 `defaultModel`。确认配置文件里有有效的 Provider 配置和 API Key。

Q3： `push_reminder` 任务触发了，但飞书没收到消息？

先确认飞书配置是否正确（见第 7 章），再查看任务运行历史里的「推送状态」字段，会显示推送失败的具体原因。

Q4：能让 Agent 「明天提醒我 X」吗？

能，用一次性 Cron（固定日期）或 `push_reminder` 类型：

明天上午 9 点提醒我看完 PyTorch 第三章

Agent 会创建一个明天 9 点的 `push_reminder` 任务，内容是你的提醒文字，触发一次后自动禁用。

Q5：任务太多，怎么快速清理？

在对话中：

帮我列出所有定时任务，删除三个月没有运行过的

Agent 会调用 `list_scheduled_jobs`，根据 `lastRunAt` 判断哪些是长期未使用的，然后逐一删除。

小结

- 任务存储在 `data/jobs/jobs.json`，运行记录追加到 `runs.jsonl`
- 调度器每 60 秒轮询，有最多 60 秒延迟；`push_reminder` 不走 Agent 循环，最快
- 7 种任务类型覆盖日常学习管理；`custom_prompt` 支持任意定制
- 时区必须显式填写（推荐 `Asia/Shanghai`），否则 Cron 按服务器时区解析
- 用「立即运行」测试任务是否正常工作，不要等到 Cron 触发才发现问题

第 7 章 即时通讯渠道接入

学完本章，你将完成飞书机器人的完整配置，在手机上直接和 Inno Agent 对话并接收学习报告推送。

学习目标

1. 理解渠道系统的统一调度架构和会话隔离机制
2. 从零完成飞书自建应用的创建和配置（含获取 open_id）
3. 掌握本地开发环境下的 ngrok 内网穿透调试方法
4. 了解微信 Bridge 模式的工作原理和配置方式
5. 理解 `personalOnly` 安全模式的必要性

7.1 渠道架构：所有入口共享同一套记忆

无论你通过 Web UI、飞书还是微信给 Agent 发消息，背后都走同一套三层记忆（L1/L2/L3）。

`PersonalChannelDispatcher` 作为统一调度层，负责：

统一处理逻辑（所有渠道一致）：

行为	说明
24 小时 TTL 去重	同一 <code>(channel, messageId)</code> 对不会被处理两次，防止飞书重发导致重复响应
<code>/new</code> 指令识别	发送 <code>/new</code> （或「新对话」等中文等价表达）自动开启新的 L3 会话
图片附件处理	图片下载后转 base64，和文字一起发给模型
消息截断	超过 20,000 字符自动截断，防止超长消息撑爆上下文

行为	说明
渠道标签注入	每条消息前附加来源标签 (<code>[feishu]</code> / <code>[wechat]</code>), 让 Agent 知道该往哪个渠道回复
会话隔离	每个渠道聊天 (飞书的一个对话、微信的一个聊天) 对应独立的 L3 会话, 不污染 Web UI 的对话历史

关键设计：渠道层是「单人」系统。没有群聊路由、没有多用户隔离，所有消息最终都指向同一个学习者画像（L1）和知识库（L2）。这个限制让系统保持简单，画像不会被多个用户的数据混淆。

7.2 飞书接入：完整配置步骤

飞书是目前稳定性最好的渠道，基于 Lark WebSocket 长连接（不需要你的服务器有公网 IP 也能接收消息）。

前置条件：

- 有飞书账号（个人或企业版均可）
- Inno Agent 服务已启动（至少能 `curl /health` 正常）

第 1 步：在飞书开放平台创建自建应用

1. 打开飞书开放平台： `https://open.feishu.cn/app`
2. 点击「创建企业自建应用」
3. 填写应用名称（如「Inno Learning Agent」）和描述，上传图标
4. 点击「确定」，进入应用管理页面

【截图占位】 此处插入截图——飞书开放平台创建应用页面，填写应用名称和描述

第 2 步：获取四项凭证

在应用管理页面，点击左侧「凭证与基础信息」：

- 记录 `App ID`（形如 `cli_xxxxxxxxxx`）
- 记录 `App Secret`（点击查看，复制完整值）

【截图占位】 此处插入截图——飞书凭证页面，框出 `App ID` 和 `App Secret` 位置

然后点击左侧「事件订阅」→「添加事件」→ 搜索并添加 `im.message.receive_v1`（接收消息事件）。

在「事件订阅」页面还可以找到 `Verification Token` 和 `Encrypt Key`（点击「重置」生成，复制保存）。

第 3 步：填写 config.json

```
{
  "feishu": {
    "appId": "cli_xxxxxxxxxx",
    "appSecret": "你的 App Secret",
    "verificationToken": "你的 Verification Token",
    "encryptKey": "你的 Encrypt Key (如果启用了加密)"
  },
  "channels": {
    "feishu": {
      "enabled": true,
      "personalOnly": true,
      "allowedUserIds": []
    }
  }
}
```

注意： `allowedUserIds` 先留空，等第 5 步拿到自己的 `open_id` 再填入。

第 4 步：配置回调地址

飞书事件订阅需要一个 HTTPS 回调地址，接收飞书服务器发来的消息事件。

方式 A：有公网 IP 的服务器（生产环境）

在「事件订阅」页面，填写回调 URL：

```
https://你的域名/api/channels/feishu/events
```

方式 B：本地开发调试（使用 ngrok）

如果你在本地运行，飞书无法直接回调到 `localhost`，需要用内网穿透工具。推荐 ngrok：

```
# 安装 ngrok (macOS)
brew install ngrok
```

```
# 或者直接下载: https://ngrok.com/download

# 启动穿透 (将飞书事件转发到本地 3000 端口)
ngrok http 3000
```

ngrok 启动后会显示一个临时 HTTPS 地址, 例如:

```
https://abc123.ngrok-free.app
```

在飞书事件订阅里填写:

```
https://abc123.ngrok-free.app/api/channels/feishu/events
```

注意: ngrok 免费版每次重启地址会变。如果需要固定地址, 考虑 ngrok 付费版或使用自己的服务器。

【截图占位】 此处插入截图——飞书「事件订阅」配置页面, 展示已填写的回调 URL 和已添加的事件类型

第 5 步: 获取自己的飞书 open_id

`open_id` 是你在这个飞书应用下的唯一用户标识, 用于白名单配置。

最简单的方法: 先把 `allowedUserIds` 留空 (允许所有人, 仅用于获取 ID), 重启服务后给机器人发一条消息, 然后查看服务器日志:

```
# 过滤飞书相关日志
npm run server -- --home ./runtime --workspace ./workspace 2>&1 | grep -i feishu
```

你会看到类似:

```
[feishu] received message from user: ou_xxxxxxxxxxxxxxxxxx
```

复制这个 `ou_` 开头的字符串, 填入 `allowedUserIds`:

```
"allowedUserIds": ["ou_xxxxxxxxxxxxxxxxxx"]
```

然后重启服务, 白名单生效。

第 6 步：申请机器人权限

在飞书开放平台左侧「权限管理」→「开通权限」，搜索并开通以下权限：

- `im:message`（接收和发送消息）
- `im:message.group_at_msg`（可选，如果需要在群里使用）

【截图占位】 此处插入截图——飞书权限管理页面，已开通 `im:message` 权限

第 7 步：重启服务验证

修改 `config.json` 后重启服务：

```
npm run server -- --home ./runtime --workspace ./workspace
```

给飞书机器人发一条消息（在飞书里找到你刚创建的应用，进入机器人对话）：

你好，帮我讲解一下什么是梯度下降

如果 Agent 正常回复，飞书接入完成。

【截图占位】 此处插入截图——飞书聊天界面，展示向机器人发消息后 Agent 的回复，可以看到文字流式输出效果

`personalOnly` 安全模式说明

`personalOnly: true` 是安全关键配置。设置后，只有 `allowedUserIds` 白名单内的用户发来的消息才会被处理，其他用户的消息会被静默丢弃（不回复、不报错）。

为什么必须开启：Inno Agent 是单人系统，一旦有其他人能和你的 Agent 对话，他们的消息会影响你的 L1 画像和 L2 知识库。生产环境务必配置白名单。

7.3 微信接入（Bridge 模式，实验性）

微信不像飞书有官方的企业机器人 API，因此通过第三方 iLink 协议客户端 + Bridge HTTP 协议实现接入。

这是实验性功能，稳定性依赖于 iLink 协议的可用性，不建议作为主要渠道使用。

Bridge 架构原理：

```

手机微信 ↔ 微信客户端 (登录状态)
      ↓
    iLink 协议 Sidecar 进程
    监听 localhost:4319
      ↓ POST /api/bridge/messages (Bearer token)
    Inno Agent (localhost:3000)
      ↓ POST sidecarBaseUrl/reply
    iLink 协议 Sidecar 进程
      ↓
手机微信 ↔ 收到回复消息

```

配置步骤 (需要第三方 Sidecar):

第 1 步: 生成随机 Bridge Token

```

# 生成一个随机密钥
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"

```

第 2 步: 填写 config.json

```

{
  "channels": {
    "wechat": {
      "enabled": true,
      "mode": "bridge",
      "personalOnly": true,
      "sidecarBaseUrl": "http://127.0.0.1:4319"
    }
  },
  "bridge": {
    "token": "你刚才生成的随机密钥"
  }
}

```

第 3 步: 启动 iLink Sidecar

参考 iLink 协议客户端的文档启动 Sidecar, 配置:

- 监听端口: 4319 (与 `sidecarBaseUrl` 一致)
- 回调地址: `http://127.0.0.1:3000/api/bridge/messages`
- Bearer Token: 与 `bridge.token` 一致

当前限制：iLink Sidecar 不包含在本仓库中，需要另行获取。QQ 渠道的 Bridge Sidecar 也尚未发布，`qq` 渠道名在代码中为保留值。

7.4 渠道使用技巧

在飞书里使用文件/图片

直接向机器人发送图片，Agent 可以理解图片内容（需要多模态模型支持）。发送文件（PDF 等）后，可以直接告诉 Agent 「帮我归档这个文件到知识库」。

`/new` 指令

在任何渠道发送 `/new`（或「开始新对话」「新建对话」等中文表达），Agent 会自动创建新的 L3 会话，清空对话历史。适用于：

- 换话题，不想让之前的对话干扰新话题
- 感觉 Agent 对当前话题的上下文已经混乱

消息长度限制

飞书单条消息有字数限制，Agent 的长回复会被自动分段发送（每段约 4000 字符以内）。如果需要看完整内容，建议在 Web UI 里查看。

7.5 常见问题

Q1：飞书机器人没有回复，也没有报错？

按以下顺序排查：

1. 确认服务在运行：`curl http://localhost:3000/health`
2. 确认飞书配置已启用：`config.json` 里 `channels.feishu.enabled: true`
3. 确认回调 URL 可达：从外网访问 `https://你的域名/api/channels/feishu/events`，应该返回 HTTP 200
4. 本地调试：如果是本地服务，确认 ngrok 正在运行，且 URL 没有过期
5. 查看日志：服务器终端日志里搜索 `[feishu]`，看是否有事件接收记录

Q2：飞书机器人回复了，但发的不是中文？

检查你发给 Agent 的消息是否正确送达（日志里应该有 `[feishu] message received`），以及模型配置是否支持中文（大多数模型默认支持，除了部分纯英文的本地小模型）。

Q3：`personalOnly` 开启后，我自己的消息也收不到回复了？

确认 `allowedUserIds` 里的 `open_id` 是否填写正确。`openid` 格式是 `ou_` 开头的字符串，不是手机号或邮箱。重新执行「第 5 步：获取 `openid`」的流程。

Q4：想让助手在飞书群里使用可以吗？

当前版本不支持群聊多人路由——所有消息指向同一个学习者画像，群聊会让多人的消息混入同一个 L1，影响个性化效果。如果真的需要，可以把 `personalOnly` 关闭，但结果是所有人都在共用一套记忆，不推荐。

Q5：服务重启后飞书就不响应了？

基于 WebSocket 长连接的飞书接入，服务重启后需要重新建立连接，通常在启动日志里会看到 `[feishu] WebSocket connected`。如果没看到这条日志，检查 `appId` 和 `appSecret` 是否正确。

小结

- 飞书原生接入基于 Lark WebSocket 长连接，不需要公网 IP 也能收消息
- 四步配置：创建应用 → 填写凭证 → 设置事件订阅 → 获取 `open_id` 填白名单
- 本地调试用 ngrok 做内网穿透；生产环境用域名直接回调
- `personalOnly` + `allowedUserIds` 白名单是必须配置的安全保障
- 微信 Bridge 模式为实验性功能，稳定性不如飞书

第 8 章 Practice Lab（实践终端）

学完本章，你将能在 Web UI 内嵌终端中运行 Agent 生成的代码，并查看完整的运行记录。

学习目标

1. 理解 Practice Lab 「写代码→运行→解释」的完整闭环原理
2. 掌握从请求演示到查看输出的完整操作流程
3. 了解终端的安全边界设计
4. 配置沙箱安全模式（生产环境必读）

8.1 Practice Lab 的设计动机

学习编程或做实验，仅靠阅读代码是不够的——你需要真正运行它、观察输出、调整参数、再观察变化。传统的工作流是：AI 生成代码 → 你复制到 IDE → 运行 → 遇到问题再问 AI。这中间有太多手动操作，而且 AI 看不到你的实际运行结果，只能猜测问题所在。

Practice Lab 的设计目标是消除这些中间步骤：Agent 生成的代码直接在同一个界面里运行，输出结果对双方都可见，后续对话可以基于真实的运行结果进行。

8.2 完整工作流程

以「我想学 PyTorch 的矩阵乘法」为例，一次完整的 Practice Lab 会话：

① 用户发出请求

帮我创建一个演示 PyTorch 矩阵乘法的练习，我想自己运行一下

② Agent 调用 `create_practice_lab`

Agent 在工作区（如 `workspace/pytorch-matmul/`）创建文件：

```
workspace/pytorch-matmul/
├── README.md           ← 说明文件：这个 Lab 做什么、怎么运行
├── matmul_demo.py      ← 主要代码文件 (mainFile)
└── requirements.txt    ← 依赖列表（如果需要）
```

工具返回：

```
{
  "mainFile": "workspace/pytorch-matmul/matmul_demo.py",
  "suggestedCommand": "python matmul_demo.py"
}
```

③ Web UI 自动响应

- 右侧工作区面板自动打开 `matmul_demo.py`，显示代码内容
- 代码查看区上方出现「运行」按钮（绿色播放键）
- Agent 在对话中说明代码的逻辑，等待用户点击运行

【截图占位】 此处插入截图——右侧工作区面板展示刚创建的代码文件，顶部有绿色「运行」按钮，左侧对话区有 Agent 的代码说明

④ 用户点击「运行」

终端（底部抽屉或右侧标签页）展开，执行 `python matmul_demo.py`，输出流式显示：

```
Matrix A (3x4):
tensor([[0.1, 0.5, ...],
        ...])
Matrix B (4x2):
...
Result A @ B (3x2):
...
```

⑤ 运行记录保存

执行完成后，完整的输出内容保存为运行记录，路径类似 `data/runs/2026-06-06/<run_id>.{json,log}`。

⑥ 用户追问

```
输出的矩阵维度是 (3, 2)，这是怎么算出来的？
```

Agent 在下一轮回复时，会把运行记录的输出尾部自动附加到上下文（通过 `before_agent_start` 钩子），能看到你刚才的实际输出，给出基于真实结果的解释，而不是猜测。

8.3 内嵌终端的使用

Web UI 有两个地方可以访问终端：

- 右侧「终端」标签页：全尺寸终端，适合长时间使用
- 底部终端抽屉：点击代码区底部的抽屉展开，不遮挡代码区

两者使用的是同一个 PTY 进程（同一会话内共享）。

终端操作：

操作	说明
普通输入	直接在终端里输入命令执行，不受 Practice Lab 流程限制
中断运行	<code>Ctrl+C</code> 中断当前运行的命令
调整字体	右上角字体大小调节按钮
清屏	<code>clear</code> 命令，或 <code>Ctrl+L</code>

终端安全限制（不可绕过）：

1. **工作目录限制**：终端 shell 启动时，工作目录设为会话绑定的工作区目录。 `create_practice_lab` 写入的文件必须在此目录内，路径包含 `..` 的访问会被拒绝
2. **环境变量清除**：启动前自动删除名称中含 `API_KEY`、`SECRET`、`TOKEN`、`PASSWORD` 的环境变量，防止意外泄露

【截图占位】 此处插入截图——底部终端抽屉展开，正在运行 Python 脚本，流式输出 `print` 内容

8.4 沙箱安全模式

沙箱模式在操作系统层面对 Agent 进行权限隔离。在沙箱开启时：

- bash 命令通过 macOS `sandbox-exec` 或 Linux `bubblewrap` 运行，文件访问受到策略约束
- 当 Agent 尝试访问沙箱策略禁止的路径时，弹出交互式提示，用户可以选择：允许本次 / 允许本项目 / 允许全局

启用沙箱：

前置条件：安装 `ripgrep`（沙箱依赖它做文件搜索）

```
# macOS
brew install ripgrep

# Ubuntu/Debian
sudo apt install ripgrep
```

然后用沙箱模式启动：

```
npm run server:sandbox -- --home ./runtime --workspace ./workspace --port 3000
```

沙箱配置文件（`runtime/config/sandbox.json`）：

```
{
  "enabled": true,
  "network": {
    "allowedDomains": ["github.com", "*.github.com", "pypi.org"]
  },
  "filesystem": {
    "allowRead": [".", "~/.config", "/usr/local/lib"],
    "allowWrite": [".", "/tmp"],
    "denyRead": ["/etc/passwd", "~/.ssh"],
  }
}
```

```
"denyWrite": [".env", "*.pem", "*.key", "~/.gitconfig"]
}
}
```

字段说明：

字段	说明
<code>network.allowedDomains</code>	允许访问的域名（不在列表内的网络请求会被阻断）
<code>filesystem.allowRead</code>	允许读取的目录/文件（ <code>.</code> 表示工作区当前目录）
<code>filesystem.allowWrite</code>	允许写入的目录/文件
<code>filesystem.denyRead</code>	明确禁止读取（优先级高于 <code>allowRead</code> ）
<code>filesystem.denyWrite</code>	明确禁止写入（保护敏感文件）

生产环境建议：`denyWrite` 至少包含 `.env`、`*.pem`、`*.key`、`config.json`，防止 Agent 意外修改配置或密钥文件。

8.5 查看运行记录

每次终端执行的输出都保存为运行记录（Run Record）。

在 Web UI 查看：

在「终端」标签页下方的「历史运行」列表，点击任意一条运行记录展开详情：

字段	说明
运行 ID	唯一标识符，用于 Agent 在对话中引用
来源文件	由哪个 Practice Lab 创建的（ <code>sourceFile</code> 字段）
命令	实际执行的命令字符串
开始/结束时间	时间戳
退出码	0 = 成功，非 0 = 失败
输出日志	完整的 stdout + stderr，最多保留 256 KB

【截图占位】 此处插入截图——「历史运行」列表，展示最近 3 条运行记录，点开一条后显示详情和输出日志

磁盘位置：

```
data/runs/
├── YYYY-MM-DD/
│   ├── <run_id>.json    ← 元数据（命令、时间、退出码等）
│   └── <run_id>.log      ← 原始输出（ANSI 转义码已清除）
```

8.6 常见问题

Q1：运行后终端没有输出？

可能原因： 1. Python 等解释器未安装——在终端手动运行 `python --version` 检查 2. 代码里没有 `print` 语句——如果代码只是计算而没有输出，终端会看起来「空」 3. 代码陷入无限循环——等待几秒后用 `Ctrl+C` 中断

Q2：终端报错「Permission denied」？

沙箱模式下文件访问被限制。检查 `sandbox.json` 的 `allowRead` / `allowWrite` 配置。如果不想用沙箱，去掉 `:sandbox` 后缀重新启动：

```
npm run server -- --home ./runtime --workspace ./workspace
```

Q3：代码文件被 Agent 创建在了工作区之外，运行按钮不出现？

`create_practice_lab` 会验证文件路径不超出工作区范围（路径不包含 `..`）。如果 Agent 写的路径有误，文件创建会失败，Agent 会收到错误信息并重试。如果仍然失败，可以直接告诉 Agent：「请把文件创建在 `workspace/` 目录下」。

Q4：关闭并重新打开终端，之前运行的环境还在吗？

关闭终端标签页或抽屉不会终止 PTY 进程，再次打开时连接同一个会话，之前安装的 Python 包、设置的环境变量仍然有效。刷新页面或重启服务才会重置终端会话。

Q5：想让 Agent 直接运行命令，不需要点「运行」按钮可以吗？

出于安全考虑，这是设计上的限制——Agent 永远不会绕过用户确认直接执行命令。如果你信任 Agent 可以直接操作，可以在沙箱配置里设置更宽松的权限，然后在终端里手动执行 Agent 建议的命令。

小结

- Practice Lab 是「Agent 写代码 + 用户点击运行 + 双方看到输出 + 继续对话」的闭环
- 终端工作目录严格限制在工作区内，安全边界不可绕过
- 运行记录持久化存储，Agent 在后续对话中可以基于真实输出解释
- 生产环境建议开启 `--sandbox` 模式，并在 `sandbox.json` 里保护敏感文件路径

第 9 章 Skills 扩展

学完本章，你将理解 Skill 的工作原理，能够安装、管理，并根据需要为特定工作区配置项目级 Skill。

学习目标

1. 理解 Skill 包的内部结构和 Pi SDK 加载机制
2. 掌握通过 Web UI 安装和管理 Skill 的完整操作
3. 理解全局 Skill 与项目级 Skill 的优先级关系
4. 了解 Skill 包的目录结构，能判断一个 Skill 包是否有效

9.1 什么是 Skill

Skill 是 Pi SDK 的扩展机制。一个 Skill 包是一个 `.zip` 文件，解压后包含一组工具定义和可选的提示词文件。安装后，这些工具会被注册到当前会话的 Agent 工具集里，扩展 Agent 的能力边界。

Skill 能做的事：

- 添加新的工具（如网页搜索工具、特定 API 调用工具、文件格式转换工具）
- 注入额外的系统提示词（如特定领域的知识背景或角色设定）
- 组合多个工具形成工作流（如「查询 + 汇总 + 写入 Wiki」组合工具）

Skill 不能做的事：

- 覆盖 Inno Agent 的核心工具（L1/L2/调度器等）
- 修改系统级配置
- 访问工作区之外的文件系统（受到 `create_practice_lab` 路径约束）

9.2 Skill 包的结构

一个有效的 Skill 包解压后是这样的目录结构（以 Pi SDK 约定为准）：

```
my-skill/  
├─ skill.json      ← 必须：Skill 元数据（名称、描述、版本）
```



```

├─ tools/
│   └─ search_web.json ← 工具定义文件（描述、参数 schema）
│   └─ ...
├─ prompts/
│   └─ system.md        ← 可选：附加到系统提示词的内容
└─ README.md           ← 可选：Skill 说明文档

```

`skill.json` 示例：

```

{
  "name": "web-search",
  "version": "1.0.0",
  "description": "为 Agent 添加网页搜索能力",
  "tools": ["tools/search_web.json"]
}

```

安装时，服务器端执行：

```
unzip <skill-name>.zip -d runtime/skills/<name>/
```

然后 Pi SDK 在下次会话启动时加载这个目录。

9.3 安装 Skill (Web UI)

步骤：

1. 打开右侧「Skills」标签页
2. 点击页面顶部的「上传 Skill」按钮（或拖拽文件到上传区）
3. 选择 `<skill-name>.zip` 文件
4. 等待上传和解压完成（通常 < 5 秒）
5. 在 Skill 列表中找到刚安装的 Skill，确认已显示
6. 点击「重新加载」按钮（页面顶部）—— 让 Pi SDK 热加载新 Skill，**无需重启服务**

【截图占位】 此处插入截图——Skills 标签页，展示已安装的多个 Skill 列表（显示名称、版本、状态开关）和顶部「上传 Skill」按钮

安装完成后，**新开一个会话**，新 Skill 的工具就会出现在 Agent 可调用的工具集里。已有的会话不会自动获得新 Skill（Pi SDK 在会话启动时加载工具）。

9.4 管理已安装的 Skills

启用 / 禁用：

点击 Skill 条目右侧的开关。禁用后，下次创建新会话时不会加载该 Skill；已有会话不受影响（Pi SDK 在会话启动时加载，不实时更新）。

删除：

点击 Skill 条目右侧的删除图标，确认后从 `runtime/skills/` 目录删除 Skill 目录，并更新列表。

查看详情：

点击 Skill 名称，展开详情卡片，显示 `skill.json` 里的描述、版本、包含的工具列表。

【截图占位】 此处插入截图——展开一个 Skill 的详情卡片，显示工具名称列表和描述

9.5 项目级 Skill

全局 Skill 放在 `runtime/skills/`，对所有会话生效。

项目级 Skill 放在工作区的 `.inno/skills/` 目录下，只对绑定了该工作区的会话生效，且优先级高于全局 Skill（同名 Skill 以项目级为准）。

使用场景：

- 你有一个专门用于 Python 数据分析的工作区，需要数据可视化工具，但不想让这个工具出现在其他学习场景的会话里
- 某个项目需要访问特定内部 API，这个工具只对这个项目有意义

目录结构：

```
workspace/
├── my-project/
│   ├── .inno/
│   │   └── skills/
│   │       ├── data-viz/ ← 解压后的 Skill 目录（不是 .zip）
│   │       ├── skill.json
│   │       └── tools/...
│   ├── code/
│   └── ...
```

注意：项目级 Skill 直接放解压后的目录，不需要上传 zip——只需在工作区里创建

`.inno/skills/<skill-name>/` 目录并放入 Skill 文件即可。

9.6 常见问题

Q1: 安装后 Skill 没有出现在列表里？

可能是 zip 文件结构有问题。检查：zip 解压后根目录是否直接包含 `skill.json`（不能有多余的嵌套层级），且 `skill.json` 的格式是否正确。

Q2: 点击「重新加载」后已有会话里还是没有新 Skill 的工具？

Pi SDK 在会话启动时加载工具，已存在的会话不会动态更新。需要新建一个会话，新 Skill 的工具才会出现。

Q3: 某个 Skill 安装后 Agent 调用工具报错？

查看服务器日志里是否有 Skill 加载错误（搜索 `[skill]` 关键词）。常见原因：工具定义的参数 schema 格式有误，或工具依赖的外部 API Key 未配置。

Q4: 我想写一个自己的 Skill，从哪里开始？

参考 Pi SDK 的文档了解工具定义格式（JSON Schema 风格）。简单来说，创建 `skill.json` + 一个工具定义 JSON 文件 + 可选的系统提示词 Markdown 文件，打包成 zip 上传即可。

小结

- Skill 是 `.zip` 打包的工具集，解压后包含 `skill.json` + 工具定义文件
- Web UI 上传后点「重新加载」，无需重启服务；但需要新建会话才能用新 Skill
- 全局 Skill 对所有会话有效；项目级 Skill（`<workspace>/inno/skills/`）优先级更高，只对绑定了该工作区的会话生效

第 10 章 进阶部署

本章适合需要在 Linux 服务器长期运行 Inno Agent，或希望用 Docker / Electron 部署的用户。

学习目标

1. 掌握生产环境路径规划原则，避免数据被代码更新覆盖
 2. 使用 systemd 将 Inno Agent 注册为系统服务，开机自启
 3. 配置 nginx 反向代理，支持 HTTPS 和自定义域名
 4. 使用 Docker Compose 一键部署
 5. 掌握备份策略和版本更新流程
-

10.1 多 Provider 配置

`config.json` 支持同时配置多个 Provider，在 `providers` 对象中添加多个 key 即可：

```
{
  "defaultProvider": "innospark",
  "defaultModel": "claude-sonnet-4-6",
  "providers": {
    "innospark": {
      "baseUrl": "https://api.innospark.cn",
      "api": "anthropic-messages",
      "apiKey": "sk-ant-xxx",
      "models": [
        {
          "id": "claude-sonnet-4-6",
          "name": "Claude Sonnet 4.6",
          "contextWindow": 200000,
          "maxTokens": 16384
        }
      ]
    },
    "deepseek": {
      "baseUrl": "https://api.deepseek.com/anthropic",
      "api": "anthropic-messages",
      "apiKey": "sk-xxx",
      "models": [
        {
          "id": "deepseek-v4-pro",
          "name": "DeepSeek V4 Pro",
          "contextWindow": 1000000,
          "maxTokens": 32768
        }
      ]
    },
    "ollama-local": {
      "baseUrl": "http://localhost:11434/v1",
      "api": "openai-completions",
      "apiKey": "ollama",
      "models": [
        {
          "id": "qwen2.5:14b",
```

```

        "name": "Qwen2.5 14B (本地)",
        "contextWindow": 32768,
        "maxTokens": 8192
    }
]
}
}
}

```

Web UI「设置」面板中可以在所有已配置的模型间自由切换，切换结果热写入 `config.json`，无需重启。这意味着你可以根据任务难度灵活选择：日常问答用本地小模型（低延迟），复杂推理换云端大模型（高质量）。

10.2 生产环境路径规划

核心原则：代码目录只放代码，数据目录与代码目录完全分离。

分离的好处：`git pull` 更新代码时不会误删数据；可以对数据目录单独做备份。

推荐的 Linux 服务器目录结构：

```

/opt/inno-agent/          # 代码仓库 (git clone 的目录)
/etc/inno-agent/
|   └─ config.json        # 配置文件 (含 API Key, 严格控制权限)
/var/lib/inno-agent/
|   └─ data/              # 运行数据 (sessions / jobs / learner / l2)
|   └─ skills/            # 已安装的 Skill 包
/srv/inno-workspace/      # Agent 工作区 (Agent 可读写的文件目录)

```

创建目录并设置权限：

```

# 创建目录
sudo mkdir -p /etc/inno-agent
sudo mkdir -p /var/lib/inno-agent/data
sudo mkdir -p /var/lib/inno-agent/skills
sudo mkdir -p /srv/inno-workspace

# 将目录权限归属给运行服务的用户 (假设用户名为 inno)
sudo chown -R inno:inno /var/lib/inno-agent /srv/inno-workspace

# 配置文件包含 API Key, 只允许 root 和 inno 用户读取

```

```
sudo chown root:inno /etc/inno-agent/config.json
sudo chmod 640 /etc/inno-agent/config.json
```

启动命令（使用环境变量）：

```
INNO_CONFIG_DIR=/etc/inno-agent \
INNO_DATA_DIR=/var/lib/inno-agent/data \
INNO_SKILLS_DIR=/var/lib/inno-agent/skills \
INNO_WORKSPACE_DIR=/srv/inno-workspace \
INNO_PORT=3000 \
npm run server --prefix /opt/inno-agent
```

10.3 使用 systemd 注册为系统服务

将 Inno Agent 注册为 systemd 服务后，可以开机自启、崩溃自动重启。

第 1 步：创建 service 文件

```
sudo nano /etc/systemd/system/inno-agent.service
```

填入以下内容（根据实际路径调整）：

```
[Unit]
Description=Inno Agent Learning Service
After=network.target
Wants=network-online.target

[Service]
Type=simple
User=inno
WorkingDirectory=/opt/inno-agent
Environment=NODE_ENV=production
Environment=INNO_CONFIG_DIR=/etc/inno-agent
Environment=INNO_DATA_DIR=/var/lib/inno-agent/data
Environment=INNO_SKILLS_DIR=/var/lib/inno-agent/skills
Environment=INNO_WORKSPACE_DIR=/srv/inno-workspace
Environment=INNO_PORT=3000
ExecStart=/usr/bin/npm run server
Restart=on-failure
RestartSec=10
```

```
StandardOutput=journal
StandardError=journal

[Install]
WantedBy=multi-user.target
```

第 2 步：启用并启动服务

```
# 重新加载 systemd 配置
sudo systemctl daemon-reload

# 启用开机自启
sudo systemctl enable inno-agent

# 立即启动
sudo systemctl start inno-agent

# 检查状态
sudo systemctl status inno-agent
```

常用管理命令：

```
# 查看实时日志
sudo journalctl -u inno-agent -f

# 查看最近 100 行日志
sudo journalctl -u inno-agent -n 100

# 重启服务（修改配置后）
sudo systemctl restart inno-agent

# 停止服务
sudo systemctl stop inno-agent
```

10.4 配置 nginx 反向代理 (HTTPS)

如果需要从外网访问（包括飞书事件订阅回调），需要配置 HTTPS。推荐使用 nginx + Let's Encrypt 免费证书。

前置条件：

- 已有域名，DNS A 记录指向服务器 IP
- 安装了 nginx: `sudo apt install nginx`
- 安装了 certbot: `sudo apt install certbot python3-certbot-nginx`

第 1 步：nginx 配置文件

```
sudo nano /etc/nginx/sites-available/inno-agent
```

```
server {
    listen 80;
    server_name your-domain.com;

    # Let's Encrypt 证书验证路径
    location /.well-known/acme-challenge/ {
        root /var/www/certbot;
    }

    # HTTP 重定向到 HTTPS
    location / {
        return 301 https://$host$request_uri;
    }
}

server {
    listen 443 ssl;
    server_name your-domain.com;

    ssl_certificate      /etc/letsencrypt/live/your-domain.com/fullchain.pem;
    ssl_certificate_key  /etc/letsencrypt/live/your-domain.com/privkey.pem;

    # WebSocket 支持（终端功能需要）
    location /api/terminal/ {
        proxy_pass        http://127.0.0.1:3000;
        proxy_http_version 1.1;
        proxy_set_header   Upgrade $http_upgrade;
        proxy_set_header   Connection "upgrade";
        proxy_set_header   Host $host;
        proxy_read_timeout  86400;
    }
}
```



```

# SSE 支持（流式对话需要）
location /api/chat/stream {
    proxy_pass          http://127.0.0.1:3000;
    proxy_http_version 1.1;
    proxy_set_header    Connection "";
    proxy_buffering     off;
    proxy_cache         off;
    proxy_read_timeout  300;
}

# 其余请求正常代理
location / {
    proxy_pass          http://127.0.0.1:3000;
    proxy_set_header    Host $host;
    proxy_set_header    X-Real-IP $remote_addr;
    proxy_set_header    X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header    X-Forwarded-Proto $scheme;
}
}

```

第 2 步：启用配置并申请证书

```

# 启用站点配置
sudo ln -s /etc/nginx/sites-available/inno-agent /etc/nginx/sites-enabled/

# 测试配置是否正确
sudo nginx -t

# 重新加载 nginx（先只启用 HTTP 部分）
sudo systemctl reload nginx

# 申请 Let's Encrypt 证书
sudo certbot --nginx -d your-domain.com

# 证书自动续期（certbot 安装时通常已自动配置）
sudo certbot renew --dry-run

```

配置完成后，飞书事件订阅的回调地址就可以使用 `https://your-domain.com/api/channels/feishu/events`。

10.5 Docker 部署

项目根目录提供 `Dockerfile` 和 `docker-compose.yml`。

快速启动：

```
# 构建镜像
docker build -t inno-agent:latest .

# 使用 docker-compose 启动
docker compose up -d
```

`docker-compose.yml` 完整示例：

```
version: "3.9"

services:
  inno-agent:
    image: inno-agent:latest
    # build: .           # 或者直接从源码构建
    container_name: inno-agent
    restart: unless-stopped
    ports:
      - "3000:3000"
    environment:
      - NODE_ENV=production
      - INNO_CONFIG_DIR=/app/runtime/config
      - INNO_DATA_DIR=/app/runtime/data
      - INNO_SKILLS_DIR=/app/runtime/skills
      - INNO_WORKSPACE_DIR=/app/workspace
      - INNO_PORT=3000
    volumes:
      # 数据持久化（必须挂载，否则重启后数据丢失）
      - ./runtime:/app/runtime
      - ./workspace:/app/workspace
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:3000/health"]
      interval: 30s
      timeout: 10s
      retries: 3
      start_period: 20s
```

运行前准备：

```
# 创建挂载目录
mkdir -p runtime/config runtime/data runtime/skills workspace

# 复制配置文件
cp config.example.json runtime/config/config.json
# 编辑配置，填入 API Key
nano runtime/config/config.json

# 启动（后台运行）
docker compose up -d

# 查看日志
docker compose logs -f inno-agent
```

重要：如果 Inno Agent 需要在容器内运行终端命令（Practice Lab），确保不在 Docker 里挂载 `/var/run/docker.sock`，避免容器逃逸风险。

10.6 Electron 桌面应用

项目支持打包为 macOS（`.dmg`）和 Windows（`.exe` / `.msi`）桌面应用，适合不想配置服务器的个人用户。

```
# macOS (Apple Silicon, 生成 .dmg)
npm run electron:build

# Windows (生成 .exe 和 .msi)
npm run electron:build:win
```

打包产物在 `dist/` 目录下。详细打包流程（含代码签名、图标配置等）参见仓库根目录的 `ELECTRON_BUILD.md`。

10.7 数据备份策略

Inno Agent 的所有数据都是普通文件，备份非常简单：直接压缩 `data/` 目录即可。

手动备份：

```
# 备份到本地
tar -czf "inno-backup-$(date +%Y%m%d).tar.gz" /var/lib/inno-agent/data/

# 备份到远程 (rsync 到另一台机器)
rsync -avz /var/lib/inno-agent/data/ user@backup-server:/backups/inno-agent/
```

定期自动备份 (crontab):

```
# 编辑 crontab
crontab -e

# 每天凌晨 3 点备份, 保留最近 30 天
0 3 * * * tar -czf /backup/inno-$(date +%Y\%m\%d).tar.gz /var/lib/inno-agent/data/ && f
```

备份优先级 (重要性从高到低):

目录	重要性	说明
data/learner/	最高	L1 画像和所有学习事件, 丢了就要重新从零积累
data/l2/wiki/	高	整理好的知识库, 手动整理成本高
data/jobs/	中	定时任务配置, 丢了可以重新创建
data/sessions/	低	对话历史, 丢了影响不大
data/l2/raw/	低	原始上传文件, 通常本地还有原件

10.8 版本更新流程

```
# 第 1 步: 停止服务
sudo systemctl stop inno-agent

# 第 2 步: 备份当前数据
tar -czf "backup-before-update-$(date +%Y%m%d).tar.gz" /var/lib/inno-agent/data/

# 第 3 步: 拉取最新代码
cd /opt/inno-agent
git pull
```

```
# 第 4 步：安装新依赖并重新构建
npm install
npm run build

# 第 5 步：重启服务
sudo systemctl start inno-agent

# 第 6 步：验证服务正常
curl http://localhost:3000/health
sudo journalctl -u inno-agent -n 20
```

10.9 运行时路径优先级速查

CLI 参数	环境变量	默认值
<code>--home</code>	<code>INNO_HOME</code>	<code>~/.inno-agent</code>
<code>--config</code>	<code>INNO_CONFIG_FILE</code>	<code><home>/config/config.json</code>
<code>--config-dir</code>	<code>INNO_CONFIG_DIR</code>	<code><home>/config</code>
<code>--data</code>	<code>INNO_DATA_DIR</code>	<code><home>/data</code>
<code>--skills</code>	<code>INNO_SKILLS_DIR</code>	<code><home>/skills</code>
<code>--workspace</code>	<code>INNO_WORKSPACE_DIR</code>	启动时的工作目录
<code>--port</code>	<code>INNO_PORT</code>	<code>3000</code>

优先级：CLI 参数 > 环境变量 > 默认值

10.10 常见问题

Q1: 服务器重启后 Inno Agent 没有自动启动?

确认 systemd 服务已启用:

```
sudo systemctl is-enabled inno-agent
# 应输出 enabled
```

如果输出 `disabled`，执行 `sudo systemctl enable inno-agent`。

Q2：升级后服务无法启动，提示数据格式不兼容？

检查 release notes，看是否有 breaking change 和迁移脚本。如果有，先执行迁移脚本。如果没有文档，先回滚到上一个版本（`git checkout <上一个 commit>`），确认数据无误后再升级。

Q3：Docker 容器里 Practice Lab 终端无法运行 Python？

容器镜像里是否安装了 Python？检查 `Dockerfile` 里是否有 `RUN apt install python3`。如果没有，需要在 `Dockerfile` 里添加，或在 `docker-compose.yml` 里挂载宿主机的 Python 环境。

Q4：nginx 配置了 HTTPS，但飞书事件订阅验证失败？

飞书验证回调时会发一个 GET 请求到你配置的 URL，检查 nginx 是否正确转发了这个请求，以及 Inno Agent 是否在运行中。可以先用 `curl -v https://your-domain.com/api/channels/feishu/events` 测试。

小结

- 数据目录与代码目录分离是生产环境的基本原则，防止 `git pull` 覆盖数据
- systemd 服务配置开机自启和崩溃自动重启，是长期运行的必要设置
- nginx 需要为 WebSocket（终端）和 SSE（流式对话）做特殊配置，否则功能异常
- 备份优先级：L1 画像 > L2 Wiki > 定时任务 > 其他
- 更新前先备份，遇到问题可以快速回滚

附录 A API 端点速查表

核心端点

方法	路径	说明
GET	<code>/health</code>	健康检查
POST	<code>/api/chat</code>	发送消息（完整响应）
POST	<code>/api/chat/stream</code>	发送消息（SSE 流式响应）

会话

方法	路径	说明
GET	/api/sessions	会话列表
POST	/api/sessions	创建会话
GET	/api/sessions/:id	会话详情
PATCH	/api/sessions/:id	更新会话
DELETE	/api/sessions/:id	删除会话
GET/PUT	/api/sessions/:id/workspace	读取/绑定工作区

学习者画像 (L1)

方法	路径	说明
GET/PATCH	/api/learner/profile	读取/更新画像
POST	/api/learner/profile/goals	创建学习目标
PATCH/DELETE	/api/learner/profile/goals/:goalId	更新/删除目标
PATCH	/api/learner/profile/knowledge/:conceptId	更新知识状态
PATCH	/api/learner/profile/misconceptions/:miscId	更新误解

知识 Wiki (L2)

方法	路径	说明
GET	/api/wiki/pages	Wiki 页面列表
GET/PUT	/api/wiki/page	读取/保存页面
GET	/api/wiki/graph	知识图谱数据
GET	/api/wiki/stats	Wiki 统计信息
POST	/api/l2/raw/upload	上传文件归档到 L2

定时任务

方法	路径	说明
GET/POST	/api/jobs	任务列表/创建任务
GET	/api/jobs/status	调度器状态概览
GET	/api/jobs/runs	最近运行记录
POST	/api/jobs/:id/run	立即执行指定任务
GET	/api/jobs/:id/runs	指定任务的运行历史
PATCH/DELETE	/api/jobs/:id	更新/删除任务

其他

方法	路径	说明
GET	/api/skills	Skills 列表
POST	/api/skills/upload	上传 Skill 包 (.zip)
PATCH	/api/skills/:name	启用/禁用 Skill
DELETE	/api/skills/:name	删除 Skill
POST	/api/terminal/sessions	创建终端会话
WS	/api/terminal/sessions/:id	WebSocket 终端流
GET	/api/settings	读取配置
GET/POST	/api/workspaces	工作区列表/创建

附录 B config.json 字段速查

字段	类型	说明
defaultProvider	string	默认 Provider key, 需与 providers 中的 key 一致
defaultModel	string	默认模型 ID

字段	类型	说明
<code>providers[*].baseUrl</code>	<code>string</code>	API 服务地址
<code>providers[*].api</code>	<code>string</code>	协议类型: <code>anthropic-messages</code> 或 <code>openai-completions</code>
<code>providers[*].apiKey</code>	<code>string</code>	API Key
<code>providers[*].models[*].id</code>	<code>string</code>	模型 ID (与服务商一致)
<code>providers[*].models[*].contextWindow</code>	<code>number</code>	上下文窗口大小 (tokens)
<code>providers[*].models[*].maxTokens</code>	<code>number</code>	最大输出 tokens, 默认 8192
<code>server.port</code>	<code>number</code>	HTTP 端口, 默认 3000
<code>channels.feishu.enabled</code>	<code>boolean</code>	是否启用飞书渠道
<code>channels.feishu.personalOnly</code>	<code>boolean</code>	是否仅允许白名单用户
<code>channels.feishu.allowedUserIds</code>	<code>string[]</code>	飞书用户 open_id 白名单
<code>feishu.appId</code>	<code>string</code>	飞书应用 AppID
<code>feishu.appSecret</code>	<code>string</code>	飞书应用 AppSecret
<code>feishu.verificationToken</code>	<code>string</code>	飞书事件订阅验证 Token
<code>feishu.encryptKey</code>	<code>string</code>	飞书事件订阅加密 Key
<code>channels.wechat.enabled</code>	<code>boolean</code>	是否启用微信渠道
<code>channels.wechat.mode</code>	<code>string</code>	<code>bridge</code> (目前仅支持 bridge 模式)
<code>channels.wechat.sidecarBaseUrl</code>	<code>string</code>	Sidecar 进程地址, 默认 <code>http://127.0.0.1:4319</code>
<code>bridge.token</code>	<code>string</code>	Bridge 鉴权 Bearer Token
<code>subagents.enabled</code>	<code>boolean</code>	是否启用子 Agent 功能

附录 C 常见问题汇总

Q：切换模型后不生效？

Web UI 切换模型会热写配置文件。如果仍不生效，检查 `runtime/config/config.json` 的文件写入权限。CLI 模式切换模型需要重启进程。

Q：Wiki 归档后找不到新生成的页面？

可能是 `wiki/index.md` 未刷新。在对话中输入「更新知识库索引」，Agent 会重新触发 `graphify_update` 重建索引。

Q：飞书消息发出去没有回复？

1. 确认 `channels.feishu.enabled: true` 且服务已重启
2. 确认事件订阅回调 URL 从飞书服务器可公网访问（本地测试需 ngrok 等工具）
3. 查看服务器日志中是否有 `[feishu]` 相关记录

Q：定时任务没有按时触发？

Cron 调度器每 60 秒轮询一次，最多有 60 秒延迟。另外检查：任务的 `enabled` 是否为 `true`，以及时区设置是否正确（建议明确指定时区，如 `Asia/Shanghai`）。

Q：Practice Lab 终端卡住没有输出？

可能是命令挂起。在终端中按 `Ctrl+C` 中断，或关闭当前终端会话后重新创建。

Q：如何完全重置学习者画像？

删除 `data/learner/profile.json` 和 `data/learner/events.jsonl`，重启服务后 Agent 会从空白画像开始。

文档版本：v0.2.3 · 更新日期：2026-06-05 · 华东师范大学上海智能教育研究院